GPSDO v1.08 — The Complete Manual (from zero to locked)

English | Polski | Español

📄 PDF: English · Polski · Español

📖 Project home · README · Changelog · Tuner

Firmware: GPSDO v1.08.71rt by jmnlabs (after André Balsa's original GPSDO, Lars Walenius' PI loop, Alan Cashin's multi-level accumulator, and the author's own Kalman filter) **Board:** WeAct BlackPill STM32F411CE · **OCXO:** 10 MHz (e.g. Vectron C4550) **GPS:** u-blox receiver on Serial1 (LEA-M8T / NEO-M8T / ZED-F9T or NEO-6M/7M)

Author: Jarosław Marek Niewiński (jmnlabs) **Assistants:** Claude Opus 5 (Anthropic) · GLM-5.3 Max (Z.ai) · Qwen3.8-Max — the Polish and Spanish translations of this manual are GLM-5.3 Max's This manual assumes **nothing**. If you have never built firmware, never flashed a microcontroller, and never used a serial terminal, start at Part 1 and do exactly what it says, in order. Every step tells you what to type and what you should see. Nothing important is left as an exercise.

> **The 60-second summary:** install Arduino IDE with the STM32 core (not 3.0.0), open the sketch, pick your display in `gpsdo_config.h`, compile with *Newlib Nano + Float printf*, flash with ST-Link or DFU. **Never do a full-chip erase** — your settings and calibration live in flash sector 7 and only a full-chip erase can destroy them. Then, over serial at 115200 baud: run `CT`, wait, run `LC`, wait, run `LA 12`, run `SAW 1`, run `ES`. Watch the phase stay near zero. Done.

Table of contents

- Part 1 — Building the firmware
- Part 2 — Flashing, and the sector 7 rule
- Part 3 — First start: calibration, algorithm, save
- Part 4 — The fourteen algorithms (0–13) and their parameters
- Part 5 — Displays: what every field means
- Part 6 — Serial telemetry (the 6-line report)
- Part 7 — Command reference (every command)
- Part 8 — The PC tuner
- Part 9 — Settings, the flash ring, and wear
- Part 10 — Troubleshooting
- Appendix A — How a GPSDO works, in plain words
- Appendix B — PID for the reluctant
- Appendix C — Glossary
- Appendix D — The Kalman filter in plain words

Part 1 — Building the firmware

1.1 What you must install

1. **Arduino IDE** (1.8.x or 2.x — both work).
2. **The STM32duino core, version 2.2.0 through 2.12.0.** Boards Manager → search "stm32" → **STM32 MCU based boards by STMicroelectronics**. Install 2.12.0 (the last supported).

> **WARNING — core 3.0.0 does not work.** It was released in July 2026 and breaks this project three ways: `ltoa()` no longer exists (compile error), `HardwareSerial Serial2(PA3, PA2)` no longer compiles, and TFT_eSPI stops driving the panel (permanent white screen). Stay on 2.12.0 or older until this project announces otherwise.

1. **Libraries** (Library Manager, install by exact name):

Library	Why
STM32duino FreeRTOS	the operating system
TinyGPS++	GPS sentence parsing
U8g2	OLED displays (only if you enable one)
Adafruit AHTX0	AHT10/AHT20 temperature/humidity sensor
Adafruit BMP280	BMP280 pressure/temperature sensor
Adafruit INA219	supply voltage/current monitor
hd44780	20x4 character LCD (only if you enable it)
TFT_eSPI	SPI TFT display

1. Nothing else. There is no EEPROM library — settings live in on-chip flash (Part 2).

1.2 Open the sketch

Unzip the project. Open `GPSDO_FreeRTOS/GPSDO_FreeRTOS.ino` in Arduino IDE. The folder must keep its name — Arduino requires the folder name to match the `.ino` file.

1.3 Pick your hardware in `gpsdo_config.h`

This one file decides what gets compiled. Lines that are active start with `#define`; lines that are off start with `//`. Uncomment exactly what your board has, comment out what it does not. The shipped configuration is a fully populated board (big TFT + all sensors); change it to match reality.

Display — pick exactly one main display:

Flag	Hardware
`GPSDO_TFT_ILI9488`	320x480 SPI TFT (shipped default, used in landscape 480x320)
`GPSDO_TFT_ST7789`	240x320 SPI TFT
`GPSDO_TFT_ILI9341`	240x320 SPI TFT
`GPSDO_OLED_SH1106` / `_SSD1306` / `_SSD1309`	128x64 I2C OLED (one only)
`GPSDO_LCD_20x4`	HD44780 20x4 via PCF8574 I2C backpack
`GPSDO_TM1637` / `GPSDO_TM1637_6`	4-digit / 6-digit LED clock (mutually exclusive, and not together with the LCD)
`GPSDO_HT16K33`	4-digit I2C LED clock (shipped default ON; independent of the main display)

Sensors and extras (all shipped ON — comment out what you don't have):

Flag	Hardware
GPSDO_AHT10	AHT10/AHT20 on I2C
GPSDO_BMP280_I2C	BMP280 on I2C
GPSDO_INA219	INA219 on I2C
GPSDO_LM75	LM75-family thermometer on I2C — LM75A/B, PCT2075, TMP75/175, TMP1075 — at GPSDO_LM75_ADDR (0x48); no library needed. Its temperature follows CPU: on the RH report's sensor line (build 68). Another chip at 0x48 can pass for one (an ADS1x15, a PCF8591): move the address or leave this out
GPSDO_VCC / GPSDO_VDD	measure the 5 V rail / 3.3 V rail
GPSDO_UBX_CONFIG	put NEO-6M/7M into the right binary mode at boot
GPSDO_FAKE_UBLOX	Chinese u-blox **clone**: baud probe only, no UBX config at all (see Part 3.2)
GPSDO_GPS_TIMING	timing-receiver support (LEA-6T/M8T/NEO-M8T/ZED-F9T): survey-in, qErr
GPSDO_PICDIV	picDIV divider arming output on PB3
GPSDO_LTIC	**the phase detector hardware** (ramp TIC on PA1) — required for algorithms 10, 11, 12. Turn it on ONLY if the detector is physically fitted: without it PA1 floats and the loop disciplines the OCXO against ADC noise
GPSDO_LTIC_ACTIVE_RESET	discharge the detector's capacitor through PA1 right after each read (shipped ON; keep it on with either detector — the classic RC Kaashoek one and the current-source variant — see Part 3.3)
GPSDO_PWM_DITHER	24-bit control voltage from a dithered 13-bit PWM (leave ON — this is the good DAC)
GPSDO_DAC_EXT	external AD5680 DAC, 18-bit, bit-bang on PB4/PB0/PB2 — can be compiled together with GPSDO_PWM_DITHER; the DAC command picks the live path at runtime. Silently disables GPSDO_TM1637/GPSDO_TM1637_6 and GPSDO_GEN_2kHz_PB5 (PB4 is the DAC chip-select)
GPSDO_SPAN_SENSE	the EFC span jumper's second pole on **PB14** (shipped ON): one CT calibration per span position, switched when the jumper moves — see SPAN in Part 7. A board without the wire reads FULL on the internal pull-up and keeps its one calibration. Turn it off only if PB14 is wired to something else on your board
GPSDO_GEN_2kHz_PB5	2 kHz test square wave on PB5
GPSDO_BLUETOOTH	HC-06 Bluetooth module on PA2/PA3
GPSDO_BLUETOOTH_PARALLEL	USB **and** Bluetooth at the same time (shipped ON)

The file refuses to compile (on purpose) if you pick two displays of the same kind or two things that claim the same pins. Read the `#error` message — it names the conflict.

Leave `GPSDO_LTIC` and `GPSDO_PWM_DITHER` on unless you truly don't have the detector hardware: without LTIC you lose algorithms 10–13, which are the whole point of v1.08.

1.4 Configure TFT_eSPI (TFT users only)

TFT_eSPI is configured **inside the library**, not in the sketch. Find `Arduino/libraries/TFT_eSPI/User_Setup.h` and make it contain the block for your panel:

240x320 (ST7789 or ILI9341):

```
#define ST7789_DRIVER          // or ILI9341_DRIVER
#define TFT_WIDTH  240
#define TFT_HEIGHT 320
#define TFT_MISO PA6      // required on STM32 even if the display has no MISO pin
#define TFT_MOSI PA7
#define TFT_SCLK PA5
#define TFT_CS   PB13
#define TFT_DC   PB12
#define TFT_RST  PB15
#define TFT_RGB_ORDER TFT_BGR
#define TFT_INVERSION_OFF       // fixes inverted colours on some ST7789 modules
#define LOAD_GLCD               // font 1 — frequency readout + splash
#define LOAD_FONT2              // font 2 — header + data grid
#define LOAD_FONT4              // font 4 — status bar, busy messages
#define SPI_FREQUENCY 40000000  // 40 MHz works; drop to 27 MHz for long wires
```

320x480 (ILI9488 / ILI9486):

```
#define ILI9488_DRIVER          // works for both ILI9488 and ILI9486
#define TFT_WIDTH  320
#define TFT_HEIGHT 480
// same TFT_MISO/MOSI/SCLK/CS/DC/RST/RGB_ORDER lines as above
#define LOAD_GLCD               // font 1 — splash credits
#define LOAD_FONT4              // font 4 — splash subtitle path
#define LOAD_GFXFF              // REQUIRED on this panel — free fonts
#define SPI_FREQUENCY 40000000  // don't skimp: this panel pushes 2.4x the pixels
```

Wire the panel: SCK→PA5, SDI→PA7, RES→PB15, D/C→PB12, CS→PB13. `TFT_MISO PA6` must be defined even if nothing is connected to it.

1.5 `build_opt.h` — leave it alone

The file contains three compiler flags and needs no action from you:

```
-DSERIAL_RX_BUFFER_SIZE=256 -DSERIAL_TX_BUFFER_SIZE=512
-DCDC_TRANSMIT_QUEUE_BUFFER_PACKET_NUMBER=16
```

The first line enlarges the serial buffers so GPS sentences are not dropped. The second grows the USB-CDC transmit queue from 128 bytes to 1 KB so the 1 Hz telemetry report (about 400 characters) always fits — without it, a host that connects but doesn't read could stall the report for seconds. The file is picked up automatically by the STM32 core. Do not delete it.

1.6 The Tools menu — board and runtime library

In Arduino IDE: **Tools →**

- **Board:** "Generic STM32F4 series" → **BlackPill F411CE**.
- **C Runtime Library: Newlib Nano + Float Printf/Scanf**. This one is mandatory. Without it the firmware compiles but prints garbage wherever a float should be (all the voltages, frequencies and phases).

1.7 Turn on USB CDC (so the board gives you a serial console over USB)

CDC is what makes `Serial` a virtual COM port on the USB cable. The boot banner, the command line, and the 1 Hz status report all travel through it. Without CDC the board still runs — the display works, the loop disciplines the oscillator — but over USB it looks completely dead: no banner, no prompt, nothing.

1. **Tools → USB support (or "USB support (if available)") → "CDC (generic Serial supersedes U(S)ART)".** The exact entry name differs slightly between core versions; pick the one that mentions **CDC** and **generic Serial**.
2. Recompile and re-flash. Changing this setting changes the firmware — it is not a runtime switch.

3. After flashing, **press the board's RESET button** once. The chip re-enumerates from the DFU/upload USB device to the CDC serial device; without the reset some hosts keep talking to the old, now-gone device.
4. A new COM port appears (Windows: "STM32 Virtual COM Port", **VID 0483, PID 5740**). If Windows shows it as "Unknown device" or "device descriptor request failed", install the driver once with Zadig — see also Part 2.5.
5. Open the port at 115200. The firmware **waits up to 3 seconds** after reset for the host to open the port before printing the banner — so a slow driver does not swallow the first lines. If you open the terminal and see nothing, press RESET again with the terminal already open.

Two more things worth knowing:

- DFU (upload) mode and CDC (run) mode are **different USB devices** to Windows. The first few upload cycles may each trigger a driver installation round — that is normal, it settles.
- With `GPSDO_BLUETOOTH_PARALLEL` compiled in (the shipped default), everything USB shows is mirrored to the Bluetooth UART at **57600** — a ready-made fallback console if USB ever fights you. The 1 Hz report goes to each port on its own (build 68), so a USB host that is connected but not reading costs at most that report's USB copy — never the Bluetooth one — and never stalls the board.

1.8 Compile and check the size

Sketch → Verify/Compile. When it finishes, read the line `Sketch uses NNNNNN bytes`.

> **NNNNNN must stay below 393216.** That is 384 KB — everything from there up belongs to the settings ring in sector 7 (Part 2). Ignore the percentage the IDE prints: it is calculated against the full 512 KB chip, so a build that "fits" by percentage can still overrun into sector 7. v1.08 builds around 300 KB (build 71: 303 364 bytes) — plenty of headroom, just don't ignore a sudden jump.
>
> **Which binary is actually running?** The banner prints the compile time — and that time belongs to the **sketch**, not to the firmware. The Arduino builder does not recompile a translation unit whose sources have not changed, so editing `GPSDO_algorithms.cpp` and uploading leaves the sketch's object file untouched with an older date inside it. The banner also prints an `image CRC32`, computed at boot from the flash itself: it cannot come out of a build cache, it changes if any byte of any translation unit changed, and two boards flashed with the same binary print the same number. When a log and a memory disagree, that is the one to trust. `V` prints it again at any time.
>
> If you want the timestamp to be right as well, run `python3 tools/bumpbuild.py` before compiling — it touches `build_id.h`, which the sketch includes for no other reason, and that makes the builder recompile the sketch and nothing else. The serial appears in the banner as `build N`.

Part 2 — Flashing, and the sector 7 rule

2.1 How the 512 KB flash chip is divided

Sectors	Address range	Contents
0 – 5	0x08000000 – 0x0803FFFF	**Your firmware** (384 KB)
6	0x08040000 – 0x0805FFFF	spare (growth buffer)
7	**0x08060000 – 0x0807FFFF**	**the flash ring: settings + calibration + learned data**

Sector 7 holds everything the device has learned about itself:

- your saved settings (algorithm choice, all loop parameters, timezone),
- the `CT` oscillator-sensitivity calibration,
- the `LC` phase-detector calibration,
- the learned drift/damping model and the last operating point.

Rebuilding all of that takes an hour of bench time. Protecting sector 7 is therefore **the single most important operational rule of this project**.

2.2 The golden rule

> **Normal uploads never touch sector 7. Full-chip erase destroys it.**
>
> - Arduino IDE upload, DFU, and the STM32duino bootloader reprogram only sectors 0–5. Your settings survive every routine firmware update.
> - "Erase Chip" in ST-LINK Utility / STM32CubeProgrammer, or `erase` without an address range in J-Link, wipes the whole chip **including sector 7**. Never use it on a working, calibrated unit.

If you use ST-Link or J-Link tools to flash manually, erase **only** the firmware range and then load:

```
> erase 0x08000000 0x0803FFFF
> loadbin firmware.bin 0x08000000
```

If sector 7 ever does get wiped: nothing breaks. On the next boot the firmware detects a blank ring, formats it, and starts from defaults — you simply re-run `CT`, `LC`, set your algorithm and timezone, and `ES`. You lose the tuning, not the device.

Before the very first flash of a finished unit, make a full backup (J-Link example; ST-LINK Utility has the equivalent "save" button):

```
> JLinkExe -device STM32F411CE -if SWD -speed 4000
> savebin backup_full.bin 0x08000000 0x80000
> exit
```

To restore: `loadbin backup_full.bin 0x08000000`.

2.3 Flashing method A — ST-Link (simplest)

1. Connect ST-Link V2 to the BlackPill SWD pins (SWDIO, SWCLK, GND, 3V3).
2. Arduino IDE → Tools → **Upload method: "STM32CubeProgrammer (SWD)"**.
3. Press Upload. Done. The programmer only writes what the sketch occupies.

2.4 Flashing method B — DFU over USB (no programmer)

> **Which BOOT0 do you have?** Older WeAct BlackPills carry a BOOT0 jumper; current boards (v3.1) have only a **BOOT0 push button**. Use the matching recipe below.

Jumper boards: put the BOOT0 jumper to **1**, press RESET — the board enumerates as "STM32 BOOTLOADER". After the upload, return the jumper to **0** and press RESET.

Button boards (v3.1), the reliable way: disconnect any external power; **press and hold BOOT0**, plug the USB cable in **keeping the button pressed**; after a few seconds release BOOT0 — Windows should raise no "unrecognised device" alert and STM32CubeProgrammer sees the board. Upload, then press RESET. (Holding BOOT0 while tapping NRST, as suggested elsewhere on the internet, usually does *not* work — especially with the board already mounted on a PCB.)

Windows may ask for a driver the first time — see the note below.

2.5 After any flash: the USB note

- After the IDE finishes, **press the board's RESET button** — the USB device re-enumerates cleanly from bootloader mode to the firmware's CDC serial.
- The firmware's USB serial is **VID 0483, PID 5740** ("STM32 Virtual COM Port"). If Windows refuses to open it, install the driver once with Zadig.

Part 3 — First start: calibration, algorithm, save

You need a terminal program (PuTTY, Terra Term, the Arduino IDE serial monitor, or the tuner from Part 8 — the tuner is the most comfortable).

3.1 Connect

1. Connect USB (and/or Bluetooth at 57600 — both work simultaneously in the shipped build).
2. Open the port at **115200, 8N1**. Line endings: either CR or LF works.
3. Press **Enter**. You should see a prompt or the 1 Hz report start.
4. Type `H` and Enter. The full command list scrolls by — that is Part 7 of this manual, live.

The boot log, line by line. The first ten seconds after reset print a fixed sequence. Here is a typical one from a healthy, already-configured unit (exact lines depend on your hardware), followed by what every line means:

```
================================================
GPSDO v1.08.71rt compiled 2026-10-29 23:34:12  build 71
image CRC32 8EED4610  (303364 bytes of flash)
FreeRTOS port by J. M. Niewinski  with Claude, GLM-5.3 Max & Qwen3.8-Max AI
https://github.com/jmnlabs/GPSDO_FreeRTOS
Inspired by GPSDO v0.06c by Andre Balsa
https://github.com/AndrewBCN/STM32-GPSDO
Algos 0-2 original design by Andre Balsa
Algos 3-9 by J. M. Niewinski
Algo 10 (LTIC 3-stage) inspired by Dan Wiering's measurements
Algo 11 (LTIC-Lars) after Lars Walenius' PI loop
Algo 12 (multi-level accumulator) after Alan Cashin (MIS42N)
Algo 13 (Kalman filter) by J. M. Niewinski - original to this project
Type H = help  SW = stack diagnostics
================================================
Reset cause: POWER-ON/BROWN-OUT PIN/NRST
Flash ring: sector 7 ready
Settings: recalled from flash ring
Live store: LRN + LC applied from flash ring
Initial PWM=44653 algo=12 time_offset_min=120
GPS init: probing baud rate...
GPS detected at 38400
GPS: disabling noisy NMEA at 38400 baud
GPS: 4/4 NMEA sentences disabled
GPS: sending UBX config at 38400 baud
UBX: CFG-NAV5 ACK
LEA-T: accepted CFG-TMODE2 (28B)
Hardware configured, creating RTOS objects...
Timers started
Starting FreeRTOS scheduler
HW: AHT10/AHT20 sensor    OK  (I2C 0x38)
HW: BMP280 sensor         OK  (I2C 0x77)
HW: INA219 sensor         OK  (I2C 0x40)
HW: LM75 sensor           OK  (I2C 0x48)
HW: LTIC phase input      enabled (PA1) - needs the ramp detector hw
TFT: init start (SPI1 PA5/PA7, CS=PB13 DC=PB12 RST=PB15)
TFT: freq-band sprite (4-bit) created
TFT: header sprite (4-bit) created
TFT: data sprite (1-bit) created
```

After that the 1 Hz report starts (Part 6). What each line tells you:

Line	Meaning
banner (`GPSDO v1.08.71rt ...`)	firmware identity — release v1.08, build 71 (`.71rt`). If you see garbage characters instead: wrong baud rate — use 115200

Line	Meaning
`image CRC32 …`	a checksum of the program in flash, computed at boot: it names the binary that is actually running, even when the compile time above is stale (see 1.8)
`Reset cause:`	why the chip restarted. `POWER-ON/BROWN-OUT PIN/NRST` = normal power-up or reset button. `SOFTWARE` = a reset commanded by firmware (`RB`, end of an upload). `INDEP-WDG/WINDOW-WDG` = a watchdog (this firmware runs none — treat as a fault signal). The extra line `-> supply dipped: check the 3V3 rail under load` appears after a brown-out: your 3V3 supply sagged under the OCXO load — fix the power supply, don't ignore it
`Flash ring: sector 7 ready`	the settings ring in sector 7 was found valid
`Flash ring: sector 7 blank/formatted (defaults)`	virgin or wiped ring — **normal on the very first flash**; the firmware formats it and uses compile-time defaults
`Settings: recalled from flash ring`	your saved settings (algorithm, loop params, timezone, flags) were applied
`Settings: none stored (compile-time defaults)`	nothing saved yet — expected on a new unit; run through Part 3 and `ES`
`Live store: LRN + LC applied from flash ring`	learned data (LC calibration, drift model, last operating point) applied — this line only appears once you have a calibrated unit
`Initial PWM=… algo=… time_offset_min=…`	the operating point chosen at boot: final PWM code (live data overrides settings when fresher), the restored algorithm, and your timezone offset in minutes
`GPS init: probing baud rate...` / `GPS detected at …`	the receiver is found and its baud measured (it prefers 38400)
`GPS: no response to baud probe, defaulting to 9600`	**no receiver answered** — check wiring to the GPS module before anything else
`GPS: disabling noisy NMEA…` / `GPS: sending UBX config…` / `UBX: CFG-NAV5 ACK`	the receiver is switched to binary mode and stationary dynamics; `NAK`/`no response` variants are survivable — the receiver just keeps its current configuration
`LEA-T: accepted CFG-TMODE2`	a timing receiver (LEA-M8T class) was configured into survey-in / Time Mode — appears only with `GPSDO_GPS_TIMING` and a timing receiver
`Hardware configured…` / `Timers started` / `Starting FreeRTOS scheduler`	internal bring-up; all three should always appear
`HW: <sensor> OK (I2C …)` / `HW: <sensor> not found`	the I2C bus scan: each sensor reports present or not. A "not found" sensor is not fatal — the unit runs without it, that field just stays empty in reports
`HW: LTIC phase input enabled (PA1)`	the firmware was BUILT with the phase detector path and PA1 is configured for it — it cannot tell a fitted detector from a floating pin, so this is not a hardware check (algorithms 10–13 depend on the real detector)
`TFT: init start (…)`	display bring-up begins. **If nothing follows this line**, the TFT wiring or `User_Setup.h` is wrong (Part 1.4) — that is the "white/dead screen over serial is fine" case
`TFT: … sprite FAILED — direct-draw fallback`	the display works but with slower redraw (out of RAM for the flicker-free sprites) — cosmetic, not fatal

Lines that appear **later**, while the unit runs, and what they mean:

Line	Meaning
`LTIC: running UNCALIBRATED (run LC)`	an algorithm 10/11/12 is active but `LC` has never run — the phase numbers are nominal, not measured. Run `LC`
`LTIC ACQ: polarity unset — run 'LPOL -1' (or +1)`	algorithm 10 refuses to steer until you set the polarity (it holds safely instead of guessing). Algorithms 11 and 12 used to assume **+1** and steer anyway; they now hold the same way
`picDIV: armed (output stopped, waiting for 1PPS sync)`	the divider was armed; a 1–1.2 s gap in its output is expected before it syncs to the next PPS edge
`picDIV: no GPS fix or no 1PPS - arming deferred until both are back`	an arm was asked for without a fix, or without a pulse in the last 1.5 s (build 68): the request waits, and is carried out once both are back
`GPS 1PPS lost — auto-holdover engaged` / `GPS fix lost — auto-holdover engaged`	the reference went — no pulse for 10 s, or the fix — and the loop froze itself (A.5); `GPS fix recovered — auto-holdover disengaged` follows once the fix is back with 10 s of unbroken pulses. Under `MH` the line ends `manual holdover kept` instead, and the holdover stays yours (build 68)
`LEA-T: … survey … %/s) — continuing anyway`	survey-in progress monitor; "continuing anyway" means the survey hit the time limit above the accuracy target — usually a blocked sky view
`!!! FreeRTOS …` (configASSERT / STACK OVERFLOW / MALLOC FAILED)	a firmware fault was caught, with file/line or task name — the blue LED blinks rapidly at the same time. Note both down; the unit needs a reset (`RB`)

A virgin unit automatically requests a calibration on the first boot — you may see the calibration countdown before anything else.

3.2 Let the GPS settle

- Give the antenna a clear view of the sky. A window sill works; a geodetic antenna on a roof works better.
- With a timing receiver (M8T/F9T...) and `GPSDO_GPS_TIMING` compiled in, **survey-in** runs at boot: the receiver measures its own position for at least 300 s until the estimate is better than 5 m, then switches to fixed-position **Time Mode**. From then on the display shows `HDOP:TIME` instead of a number. Survey-in needs to complete only once; it re-runs after power loss. (`SV 0` disables it, `SV 1` re-enables, next boot.)

Making a survey permanent. The 300 s / 5 m survey the firmware runs at boot is a compromise: long enough to be useful, short enough that nobody waits for it. If you can leave the receiver running for hours, do the survey once in u-center and save it to the module's battery-backed configuration instead — u-center **V8.29** has the right commands (UBX-CFG-TMODE2 to run the survey, then UBX-CFG-CFG → *Save current configuration*). A long survey gives a better fixed position, and it survives power cycles: the firmware checks Time Mode at boot and skips its own survey when the receiver already reports one. Alan Cashin's recommendation, and the cheapest accuracy in the whole build. - The OCXO warm-up takes 300 s (`WU 0` skips it; leave it on). - The yellow LED is **off without a GPS fix, steady on with a fix** — glance there first when something looks wrong.

Which GNSS module? A genuine u-blox timing receiver (LEA-M8T class) is what this firmware is built around: survey-in, Time Mode, the `qErr` sawtooth correction. It will also discipline from cheap navigation modules — including the Chinese u-blox clones common on eBay/AliExpress — because the 1PPS pulse and the NMEA sentences are all the loop strictly needs. But clones ignore the binary configuration: expect `0/4 NMEA sentences disabled`, unacknowledged CFG frames and a ~15 s slower boot while the timeouts expire; there is no survey-in and no `qErr`, the display shows a numeric HDOP forever, and the extra position wander lands on the phase — the loose default algo-12 limits absorb exactly that. One known clone quirk: after the failed configuration round some modules stop sending data and the display stays on "acquiring" — the cure is to enter the u-center tunnel (`T` with the module's baud, e.g. `T 9600`) and simply let it time out; the re-probe that follows restores operation. A firmware switch covers all of it: define **`GPSDO_FAKE_UBLOX`** in `gpsdo_config.h` and the firmware probes the baud rate and sends nothing else — one switch, regardless of any other GPS options enabled. Not yet hardware-tested (no clone module on the author's bench); field reports welcome.

3.3 Calibrate — `CT` first, then `LC`. The order matters.

These two commands teach the firmware two different physical facts, and the second needs the first:

Step 1 — `CT` (oscillator sensitivity + loop tuning, ~3 minutes; ~11 on a gentle plant). Type `CT` and Enter. The firmware drives the control voltage to three points (1.5 V, 2.0 V, 2.5 V), measures the frequency at each, fits a straight line, and computes **K — how many hertz one PWM step is worth on *your* oscillator** (accepted range 0.01–2 mHz/LSB). From K it then derives sensible PID gains for algorithms 3–9 and the LTIC loops, and **auto-saves the PID group** to flash. On a gentle plant — a span-reduced EFC or an external DAC, K below 0.12 mHz/LSB — a second pass re-measures around the 10 MHz point with 100-second averages, which adds about eight minutes. Do not power off while it runs. On a board with the span jumper (PB14), run it in the position you use; the other position gets its own `CT` by itself the first time the jumper goes there.

Step 2 — `LC` (phase detector calibration, ~5 minutes). Type `LC` and Enter. The firmware arms the picDIV divider, centres the detector, sweeps one capacitor ramp, and measures the detector's **ns per volt**, **zero-offset volts** and **range in ns**. On PASS it **auto-saves**. It refuses to be useful if `CT` hasn't run — that is why the order matters. If the divider lands the phase below the detector's band, `LC` waits for its own sweep to carry it in (build 68: up to two minutes, `LC: ramp wait … s` every 10 s, a second arm if that runs out), so on such a board it takes a little longer. It arms only with a GPS fix and a 1PPS pulse to sync the divider to: without them it stops with `LC: ERROR no GPS fix or no 1PPS to sync the picDIV to — aborting.` and puts the loop's code back — check the antenna and run it again.

Sanity check: type `LL` and confirm the LTIC block shows non-zero `ns_per_volt`, `zero_offset`, `range_ns`. Type `DAC` and it will even tell you the size of one output step in microhertz.

How a reading is made (build 62). Just before each PPS edge the firmware stops everything else on the chip — interrupts too, for some fifteen microseconds around the edge — and from 5 µs after the edge takes 64 conversions of the capacitor straight from the converter's registers, about 110 µs in all; it fits a line through them against their times on TIM2 and reads the line at the edge itself, before any of the charge has leaked. `LB` shows the last burst: "taken at the edge", 64 falling codes, the line, the spread about it (2–3 codes rms is normal), how fast it falls as a time constant (around 1.4 ms — that is mostly the converter itself: each conversion takes about 0.12 % of the charge, ~3 codes; the leak alone is far slower) and its bend next to the board's typical one (about −3 codes on the author's board). Every burst bends a little, the same way each second; one is refused only if it bends 3.5 codes or more unlike the board's. A conversion off the line by more than six codes (and four MADs) is left out and the line refitted, up to twenty times since build 70 (twelve from build 67, eight before): a detector that picks up a square wave gets a pair of glitches at each of its edges — on Dan's board four or five pairs a burst, about 21 µs apart, from something outside the GPSDO switching at roughly 23 kHz (they stay with the PWM pin held low and come and go with the time of day) — and with twelve a burst carrying six pairs or more was refused. Many bursts refused as "off the line" in `LB` point to such a disturbance on the detector. Since build 71 a burst with every third conversion off together has that third left out whole first, before any single conversion is dropped: on some bursts the conversions follow each other every 16⅔ periods of the 10 MHz (on others every ~17.7), so every third one meets it at the same point of its period, and a detector that picks up its edges has one third of the burst off by the same few codes — on Dan's board 17–30 codes high, or 7.5 low. Such a third pulled the line several codes off, or had more than twenty conversions off the line one by one, and the burst was refused. It is left out when it is more than 6 codes and three times the burst's own noise from the other two, those two agree, and two in three of its conversions are at least half that far out; the twenty drops stay for the glitches. `LB` then says `every third conversion from #0 left out: +29.6 codes off the other two`, and otherwise how far the furthest third is (`thirds kept: the furthest -0.5 codes off the other two`). The board's typical bend is learned only from bursts whose line reads inside the band `LC` measured: a detector against its rail (picDIV not yet synced, the phase out of the window) gives bursts bent by some +70 codes, and in build 64 they taught it +20, after which the board's own bursts were refused (fixed in build 65). And since build 71 it keeps up when the bend moves: on the author's board the bend grows as the reading falls, about 0.19 codes per code, so a step that takes the phase 60–100 ns away for minutes bends every burst unlike the typical one, which followed at a thirty-second of a code a second at most — 131 bursts refused as bent in 16 minutes of the two steps of 27.10. Now, when the median bend of the last 15 bursts in the band is more than 3.5 codes from the typical one, the typical one moves there at once (in a model of those minutes, 14–21 refused instead of 270–290); a bend that stays put is followed as before. Right after the burst the capacitor is discharged through PA1 (`GPSDO_LTIC_ACTIVE_RESET`), so the next pulse charges it from 0 V whatever the temperature; the leak alone, slower on a cold board, could leave part of one second's charge under the next reading. Before the burst it prints counts since boot:

whether the edge was caught, whether the line stood (and if not, why), and how often the last reading was kept — which it is, with the receiver's qErr that came with it, for up to 3 s when the edge was not caught or the line was refused, and when the outlier gate holds a jump back; since build 71 also how many bursts had a third left out and how often the typical bend moved at once (`every third conversion left out 12, typical bend moved 3`). `LRM 0` reads the median of sixteen slower conversions instead, as builds up to 60 did (for comparison; not saved). The two modes read the leak at different moments, so an `LC` calibration belongs to the mode it ran in: after updating to build 62, run `LC` once more.

While a burst is due — from 150 ms before each edge until 2 ms after it — the command line holds its output: text going out on the Bluetooth port at the edge put that second's reading up to ~20 ns high. A long listing such as `LB` therefore comes out with a short pause each second; nothing is lost.

3.4 Choose the algorithm

- **`LA 12`** — the multi-level accumulator (after Alan Cashin). The proven choice: holds phase to a few ns RMS, corrects on average every few minutes. **This is the recommendation.**
- **`LA 11`** — Lars Walenius' continuous PI loop. The mature, gentle classic; excellent long-term behaviour with one knob (`LTC`).
- **`LA 13`** — the Kalman filter (Part 4.6). The newest, and the best in variances rather than from a constant, and takes every number it needs from `CT` and `LC`. Nothing to tune. **The simulator flattered it** — on the real bench it is limited by the detector's own slow zero wander and lands behind algorithm 11 on the phase-reading metric (see 4.6a for why that metric may be unfair to it, and what the open question is). Newer than 12, fewer hours on hardware — the recommendation above stands; 13 is the one to watch.
- `LA 10` — three-stage phase loop (ACQ→DPLL→LOCK). Solid, more parameters.
- Algorithms 0–9 are the historical collection — they work, they are frozen, see Part 4.

`LA` alone shows the current algorithm. Selecting does **not** save — see step 3.6.

3.5 Recommended extras

- **`SAW 1`** — sawtooth correction. The GPS receiver's 1PPS quantisation error (±8…±25 ns, reported every second as `qErr`) is subtracted from the measured phase. With a timing receiver this is free accuracy; turn it on.
- **`TZ <city>`** — local time with DST (e.g. `TZ Warsaw`, `TZ Adelaide`), or `TO 1` for a fixed offset, `LT 1` to show local instead of UTC. `H TZ` explains the rule format if your zone is missing.
- **`PO <Pa>`** — offset added to the BMP280 pressure reading (−5000..5000).
- **`AO <m>`** — offset added to the **GPS altitude** at display/telemetry (−3000..3000 m). This corrects the displayed altitude to your real elevation, e.g. if the geoid model puts you 30 m off: `AO 30`.

3.6 Save everything — `ES`

Type `ES` and Enter. This writes **all** settings plus the live (learned) data to the flash ring in sector 7. From this moment a power cut loses nothing.

A handful of preference commands save **themselves** the moment you set them and tell you so — the reply says `[auto-saved: …]`. These are: `TZ`, `TO`, `LT` (timezone group), `WU`, `SPL`, `SV` (flags), `PO`, `AO` (offsets), plus `CT` (auto-saves the PID group it just tuned) and a passing `LC` (saves itself to the live ring).

Everything **loop-related** — gains (`KP/KI/KD/IL`), all LTIC and Lars parameters, the algo-12 settings, and the `LA` algorithm selection itself — lives in RAM only until you save it. Every such command answers with a hint like `[not saved – run 'ES LTIC' to keep it]`: run the named group save, or just remember: **after a tuning session, `ES`**.

3.7 What "healthy" looks like

- The trend word (display + `PWM:` report line) reaches **`LOCK`** and stays there — with algo 12, brief `CORR`/`ZC` flashes are *normal and healthy* (a correction is the algorithm self-testing on schedule; the display still counts them as locked).
- The phase (`dph:` in the report) wanders around zero within tens of ns and always comes back — it must not ramp away.

- The frequency windows close in: `100s:` in low millihertz, `1ks:` in microhertz, over an hour or two.
- `CS` (correction statistics) prints small, steady RMS numbers that don't grow hour over hour.
- On the TFT the big frequency number is **green**.

Expect, on a quiet bench with algo 12: phase 5–20 ns RMS, corrections of a few LSB every few minutes, Allan deviation in the 1e-11…1e-12 class from 1000 s upward. Near a heater, an openable door, or direct sun the numbers will be worse — that is physics, not a bug.

Part 4 — The fourteen algorithms (0–13) and their parameters

One idea underlies everything: the firmware counts the OCXO frequency with a gated counter (TIM2), measures the GPS-vs-OCXO phase with the LTIC detector, and adjusts the control voltage (PWM "DAC", 65536 steps, ~48.8 µV per step at 16 bit; with `GPSDO_PWM_DITHER` the effective resolution is 24 bit — about 0.2 µV per step). Sign convention: measured error `e = freq − 10 MHz`; if `e > 0` the oscillator is fast and the PWM must go **down** (for a positive-sensitivity EFC; `LPOL` / polarity handling covers inverted EFCs).

> If phase, frequency error, or PID are new words to you, stop here and read Appendix A (what the loop is chasing and why it must be gentle) and Appendix B (what the letters P, I and D actually do). Ten minutes there will make every parameter below readable.

The `Learn:` line in the report names the active algorithm; the tuner picks its plot family from it automatically.

4.0 The menu

#	Name (as shown)	One-liner	Status
0	primitive	André Balsa's original stepped controller, 429 s cycle	default after cold flash
1	forced-drift	+1 LSB per 1000 s — oscillator characterisation	diagnostic
2	random-walk	±1 LSB noise every 5 s — noise-floor measurement	diagnostic
3	FLL-PID-man	PID on the 100 s frequency average	classic
4	PLL-PI-man	PI on phase, 10 s cycle	classic
5	PLL-PID-man	as 4 with its own tuning slot	classic
6	FLL-PID-gen	FLL PID, genetically-derived tuning	classic
7	PLL-PID-gen	the old workhorse PLL; its Kp stores the CT result	classic
8	hybrid-FLL-PLL	sigmoid blend of 6 and 7 by error size	classic
9	NN-MLP	small neural net + learned temperature steering in holdover	classic
10	LTIC-3stage	ACQ→DPLL→LOCK state machine on the phase detector	recommended line
11	LTIC-Lars	Lars Walenius' continuous PI on phase	recommended line
12	multi-level	Alan Cashin's multi-level accumulator	**the recommendation**
13	kalman	three-state Kalman filter — phase, frequency, aging	newest

Algorithms 0–9 are **frozen**: they remain because they work and because algorithm 7's tuning slot doubles as the storage of the CT calibration. New development is 10/11/12/13 only.

Select with `LA <n>` (persists via `ES ALGO`). A fresh unit always boots algorithm 0 — select and save yours once, and it sticks forever after.

4.1 Algorithms 0–2 (diagnostic)

No parameters. 0 steps the PWM when frequency averages cross fixed thresholds. 1 ramps the PWM linearly (characterise the EFC). 2 injects noise (measure the loop's floor). You will not need them for normal operation.

4.2 Algorithms 3–9 (the classics) — parameters via KP/KI/KD/IL

`KP <algo> <val>` / `KI` / `KD` set the gains (algos 3–7, value 0..100000); `IL <algo> <val>` sets the integrator clamp (algos 3–9, 100..100000). `LP [n]` lists. Defaults (also what `CT` derives from, then overwrites with measured-based values):

algo	Kp	Ki	Kd	I_LIMIT
3	70.0	0.70	175.0	9000
4	1000	0.020	2.0	7000
5	1000	0.020	2.0	10000
6	205	0.264	14950	13000
7	1000	0.020	2.0	10000
8	—	—	—	13000
9	—	—	—	450

Extras: algorithm 8 has `BC` (blend crossover, default 0.024 Hz) and `BS` (blend scale, 0.012 Hz) — the sigmoid centre and width that decide how much FLL vs PLL is mixed in at a given error; algorithm 9 has `NS` (max step, default 175 LSB). Save any of this with `ES PID`. Honestly: after `CT` you should not need to touch these — that is what CT is for.

4.3 Algorithm 10 — LTIC three-stage

A state machine on the phase detector: **ACQ** (frequency-led pull-in, also re-centres the detector ramp) → **DPLL** (2 s cycle, frequency + phase) → **LOCK** (corrections every `LIV` seconds using a two-window pair estimate).

Parameters (all `ES LTIC`; `LL` lists everything):

Command	Meaning	Range / default
`LAT`	ACQ phase threshold	ns, default 100
`LDT`	DPLL→LOCK drift threshold	1e-13..1.0, default 5e-10
`LIV`	LOCK correction interval	1..600 s, default 300
`AQP/AQI/AQD/AQL`	ACQ-stage PID	0..100000
`DPP/DPI/DPD/DPL`	DPLL-stage PID	0..100000
`LKP/LKI/LKD/LKL`	LOCK-stage PID	0..100000
`LPOL`	PWM→phase polarity, −1/0/+1	0 = not set: the loop holds
`LCV`	ACQ centring target volts	0 (= auto mid-range) .. 3.3
`ACG g [cap]`	centring drive gain [max step]	50..20000 LSB/V [5..1000 LSB]
`FA/FAD/FAL`	damping frequency-average window (both / DPLL / LOCK)	10, 100 or 1000 s

`ltic_autotune` (part of CT/LC) fills the stage gains from the measured sensitivity, so the shipped sequence already tunes this algorithm. If ACQ never leaves, check `LPOL` — the loop warns and holds until you set it.

4.4 Algorithm 11 — LTIC-Lars continuous PI

Lars Walenius' loop, ported faithfully. One PI controller evaluated every second, with an adaptive pre-filter; lock when the filtered phase stays inside `LPL` ns for `LTC × LPF` seconds. Railed detector → automatic frequency-led pull-in, one picDIV re-arm allowed.

Command	Meaning	Range / default
`LG`	loop gain. **0 = auto from CT** (recommended)	0..10000
`LD`	damping	>0..1000, default 3
`LTC`	loop time constant — **the one knob**	1..600 s, default 60
`LFD`	pre-filter divisor (pre-filter = LTC/this)	1..100, default 2
`LTO`	phase target on the detector	0..3.300 V, default 2.111 V

Command	Meaning	Range / default
`LPL`	lock phase window	1..10000 ns, default 100
`LPF`	lock hold factor (hold = LPF × LTC)	1..100, default 5
`LTK`	temperature feed-forward, DAC steps per ADC step	−32000..32000, 0 = off
`LTR`	temperature reference	0..3.300 V

Tuning guidance in one sentence: leave `LG 0`, set `LTC` to how gently you want the loop to steer (60 s is a good start; 240 s calms a noisy site; short only makes it noisier), and let the rest default. All `ES LTIC`. The tuner's **LTIC-Lars** tab exposes exactly these fields.

4.5 Algorithm 12 — the multi-level accumulator

Ported from Alan Cashin's (MIS42N) PIC GPSDO and refined during v1.04–v1.06. No fixed cycle time at all: **the size of the error chooses the averaging time.**

How it works, in plain words. Every second the phase reading enters a ladder of accumulators. Level *n* averages over 2^(n+1) seconds — the ladder runs 2, 4, 8, … 2048 s (11 levels). Each level holds a pair of values (A = older half, B = newer half). When a pair completes, two numbers are computed: the **slope** (B − A = frequency error over that span) and the **extrapolated phase** (3B − A = phase right now). If the phase stays inside that level's limit, the pair is summed and promoted one level up (twice the averaging). If it exceeds the limit, a correction fires immediately, the ladder resets, and the correction is spread gently over the span it came from (minimum 64 s — "a GPSDO wants a large error corrected gently"; Alan's own floor is 16 s, chosen so that a warm-up correction cannot demand a control voltage outside the DAC's range — a smaller floor means bigger steps, a larger one means slower phase recovery). Independently, reaching level `MR` (default 7 = 256 s) forces a correction even under every limit — otherwise a slow drift would never be acted on.

Each correction has up to three parts, and this is deliberate: cancel the measured **frequency** error, move the **phase** back to zero, and never one without the other. Then a trick Alan calls essential: the deliberate slew is remembered, and on the first phase reading that crosses zero in the expected direction, exactly that slew is removed (**ZC**, zero-crossing cancellation) — leaving the oscillator at the right frequency *and* no phase error.

The zero-crossing test is armed **only after a correction that a limit triggered**, which is Alan's rule. A scheduled correction (the `MR` run level) fires on a timer with the phase wherever it happens to be, so there is no overshoot to wait for; and a ZC does not re-arm itself, because it *is* the cancellation. Arming it in either case would let an unrelated crossing minutes later pull a step out of a stale slope.

Trends specific to 12: `WAIT` (no data yet) → `SYNC` (5 s settle after the divider arms) → `FLL` (detector railed, frequency pull-in) → `NOPH` (no valid phase, PWM frozen) → `ACQ` → `LOCK` (>16 s of genuine quiet **and** frequency home; corrections and ZC do *not* reset the quiet timer — they are health, not noise). Transient `CORR` / `ZC` flashes are normal. `NoCT` means: gain is auto but `CT` has never run — run `CT`. `NoPL` means the EFC polarity is unset — the loop holds until you set `LPOL -1` or `1`.

Parameters (`ES ALGO12`; `ML` lists everything including the live 1-sigma noise estimate):

Command	Meaning	Range / default
`MG`	gain, LSB per ns of phase. **0 = auto from CT**	0..10000, default 0 (auto)
`MR`	level at which a correction is forced	0..10, default 7 (256 s)
`MM`	how a correction reaches the pin: 1 = calm (below), 0 = the original path	0..1, default 1
`MF`	where the per-level limits come from: 0=follow MG, 1=stored table, 2=sigma formula, 3=measured fit	0..3, default 0
`MFT`	with MF 3: target seconds between noise-driven corrections	0 (=3600) or 2048..65535 s
`MLP n v`	set one row of the 11-level limit table	level 0..10, value in nanoseconds

Calm mode (`MM 1`, the default from build 60). Once the phase is inside `LAT`, two things change and the hierarchy's arithmetic does not: from the 128 s level up the frequency term is taken whole instead of halved; and the output moves toward a correction at no more than 1 LSB per second (a change over 32 LSB — acquisition, the FLL, an arm — still goes at once), the zero-crossing taking back exactly the slew it applied, fraction included. Build 60 also made the levels under that boundary need three times their limit and tied the boundary to `MR`. With both, the board logged 6.1 ns at `MR 7` and 9.5 at `MR 10`, against 2.7 for algorithm 11 the same week; in a replay calibrated against those runs, dropping them gives 5.2 ns at `MR 7` and 5.1 at `MR 10` (`MM 0`: 6.1 and 5.6), so build 61 dropped both. The PWM moves a fifth as much as under algorithm 11, which still holds the phase about twice as tightly; pulses are how this algorithm moves the phase. Below `MR 6` no correction reaches 128 s and the frequency term stays halved. `MM 0` is the original path, unchanged, for an A/B on one board.

The default limit table is Alan's own (scaled from his 25 ns detector to this ~1 ns one). In nanoseconds per span it is: L0 462, L1 400, L2 331, L3 264, L4 191, L5 164, L6 126, L7 117, L8 108, L9 103, L10 63 ns. `ML` labels it **UNTUNED** (only the 128 s row was ever derived from a spec) — treat it as the proven starting point, not scripture. Its provenance, in Alan's own words, is a **worst-case reception scenario**: the numbers were calculated for a NEO-6 with an indoor antenna, whose 1PPS can be out by ±100 ns for short stretches (±30–40 ns on average). The table is loose on purpose — sized for the worst receiver the design might face, not for the LEA-M8T-class hardware this build usually has.

Which MF should you use? Field results from two boards:

- **Quiet site (indoor window antenna, stable temperature):** the sigma formula (`MF 2`) or measured (`MF 3`) works — the loop self-scales and corrects every few minutes.
- **Noisy site (workshop with a door, heater nearby, moving air):** the auto-scaling can hunt at level 0 (limits tighter than the environment's real wander). There, Alan's loose stored table (`MF 1`) was dramatically more stable. If you see constant level-0 corrections, switch to `MF 1`.

Behind both results sits Alan's stated design intent, worth knowing before you tune: "*we don't want the algorithm using limits, we want the algorithm using scheduled corrections at defined intervals.*" The run-level correction (`MR`) is meant to be the workhorse and the limits only a safety net for warm-up and reception excursions — which is exactly why his table is loose. A tight table firing constantly is, by that philosophy, working against the algorithm rather than with it.

The gain (`MG`) belongs to the **oscillator**; the limits belong to the **site noise** — that is exactly why the two are set by different commands and why `MF` lets you mix a measured gain with a hand-set table.

In the tuner: the **Multi-level (algo 12)** tab has the four scalars and the whole 11-row limit table with Send/Read/List buttons, and the live plots switch to the algo-12 family (`ph` phase error, level, correction count, sigma, ZC count) automatically.

4.6 Algorithm 13 — the Kalman filter (original to this project)

Algorithms 10, 11 and 12 all answer the same question — *how much of this second's phase reading should I believe?* — with a number decided in advance: a state machine's stage, a time constant, a level in a threshold table. This one answers it from the variances instead, and re-answers it every second.

It carries three states — **phase**, **frequency**, **aging** — with a covariance for each. It knows how noisy your detector is because it measures it, and how fast your oscillator wanders because it measures that too, so the weight it gives each reading is whatever those two numbers currently say. At short averaging times, where the detector is noisy and the OCXO is quiet, it leans on the oscillator; at long ones, where the OCXO walks, it yields to GPS.

What it costs. Three states and a *scalar* measurement, so the textbook matrix inversion is a single division: about 140 multiply-adds and 36 bytes, once a second — roughly a microsecond of the M4F. The claim that Kalman needs a Cortex-A or an FPGA is about twenty-state GNSS filters, not this one.

Holdover is free. The state carries frequency and aging with their covariances, so a lost phase is not a special case: it stops updating, keeps predicting, keeps steering. The trend shows `NOPH`.

It uses two measurements. The phase from the LTIC detector, and the frequency from the TIM2 gated counter. That second one is not a refinement: with the picDIV unsynced there is no valid phase at all, and a filter with only a phase measurement then has *no* measurement — it predicts from a state still at zero and the oscillator walks away. TIM2 is also

what makes the detector checkable: phase and frequency are the same quantity differentiated, so over a window the phase **must** move by the summed frequency error. A detector that does not move when TIM2 says it must is not measuring anything, and the loop re-arms the picDIV and steers on frequency alone until it behaves.

Everything is derived from `CT` and `LC`. Nothing in it is a constant measured on somebody else's bench: the detector's own quantum (`ns_per_volt × 3.3/4096`) sets the measurement-noise seed and its floor, half the detector band sets the cold-start covariance, the band crossed in one horizon sets the frequency one, and `R/KT³` seeds the process noise. Rerun `CT` or `LC` and the numbers follow within a second. When the filter starts it prints one line saying what it concluded:

```
KAL: from CT/LC  res 1.01ns  R0 2.52ns  Q0 0.000006600  P0 1500ns  lim 939LSB  arm<0.25Hz
```

Two process noises, not one. The clock model this filter implements carries `Sf`, the white frequency noise that shows up as a random walk in phase, and `Sg`, the frequency random walk — `Q = [[Sf·t + Sg·t³/3, Sg·t²/2], [Sg·t²/2, Sg·t]]`. Until 01.09 only `Sg` existed here, and that single omission was the whole of the over-actuation: with no `Sf`, the only way to explain a phase that moved more than predicted is to decide the *frequency* is wandering, which raises the gain that writes the frequency state, which is what the DAC follows. `Sf` is now measured rather than adapted — the phase differences at lag 1 and at lag 16 carry `sigma_R² + Sf·k/2`, so two lags separate white noise from walk where one cannot — and a difference that does not clear two sigma of its own estimator noise is reported as zero, which on a clean detector is the right answer. `KL` shows it beside `R` and `Q`.

Q adapts, but it may not outrun its horizon. The filter nudges the process noise `Q` toward whatever makes its own innovations come out the size it predicted. That has one systematic bias, and on the bench it bit: when the detector's error is *correlated* — a zero that wanders over minutes — the innovations are bigger than the filter expects for a reason that has nothing to do with the oscillator, so `Q` ratchets up second after second until the covariance has grown enough to explain them. It stops, but it stops high. On 30.08 the board was running with `Q` 195x its seed: a filter five times faster than the horizon it was told to steer over, moving the DAC 1.80 LSB per second against algorithm 11's 0.49 on the same night — and no better phase for it.

`(R/Q)^(1/3)` is a time, and it is the filter's own time constant; `Q = R/KT³` is exactly the statement *run as fast as the horizon you were given*. So the adaptation is now bounded there: **the filter may not run faster than `KT`**, with `R` taken as measured rather than as seeded — which is what makes the rule say the same thing on every board. Measured over two plants, five noise seeds and three levels of detector wander: short-tau ADEV twice better, DAC motion 2.4× lower, against 7% on phase sd. Nothing is lost by refusing to let `Q` absorb correlated detector error, because `R` already has it: `R` is measured from differences taken at a sixteen-second lag, so a zero that wanders on those scales is in the measurement noise where it belongs. Want a faster loop? Shorten `KT` — that is now the only thing that moves it. A value you pin yourself with `KQ` passes as given: the bound is on the adaptation, not on you. `KL` says `[at ceiling]` when it is sitting against it. That alone does not say `KT` is too long, though up to build 70 this manual and `KL` said so: what presses Q up comes as often from the GPS side's slow wander, or the detector's, as from the oscillator — and a faster loop copies it — and on Dan Wiering's board (28–29.10) `KT 1000` sat at the ceiling and was still better than `KT 500`: ADEV flat at $2.9{\cdot}10^{-12}$ from 40 to 400 s, against a $4{\cdot}10^{-12}$ hump at 100–150 s. Compare `KT` values by ADEV, a run at each.

Parameters — all four are optional and the defaults are the ones to use:

Command	Default	Meaning
`KR [ns]`	`0` = measure	Measurement noise. Zero means "measure it from the detector's own differences at a sixteen-second lag" — white noise and slow zero wander together, which is what you want. Set it only to pin the filter for an experiment.
`KQ [v]`	`0` = adapt	Process noise, $(ns/s)^2$ per second. Zero means "adapt it from the innovation sequence".
`KT [s]`	`100`	Phase horizon: how quickly the control nulls the estimated phase. Shorter follows GPS harder, longer leans on the oscillator. It also sets the patience of the stall check — five horizons far out and the picDIV is re-armed. A cold start uses at most 100 s of it since build 67 (see `KT` below).

Command	Default	Meaning
`KL`	—	List the state: phase, frequency and aging, how well each is believed, the R and Q in use, and how many readings the innovation gate threw away. `[at ceiling]` next to Q means the adaptation is against its bound.

`KR`, `KQ` and `KT` are saved the moment you type them, in their own flash-ring record — no `ES` needed, and an older firmware simply never asks for that record.

The Learn line reads

```
Learn: algo=13 (kalman) ph=-12.4ns f=-118.30ps/s sig=2.48ns R=2.64 rej=3 arm=1
```

— the phase and frequency the filter *believes* (not this second's reading, which is the whole point of having a filter), how well it believes the phase, the measurement noise it has measured, how many readings the innovation gate rejected, and how many times it has re-armed the divider. `HOLD=<s>` appears when it is running on the model alone, and `KF` while a disturbance is being nulled at the `KF` horizon (build 69).

In the tuner: the top plot shows the filter's phase estimate with `Vphase` and its band guides underneath — because the question this loop most often raises is whether the detector is alive at all.

Measured on the simulator (`tools/loopsim`, five noise seeds, two plants, white detector noise): phase sd **0.81 / 1.20 ns**, against 3.07 / 4.20 for algorithm 12 and 3.62 / 7.39 for algorithm 11. **The bench disagrees, and the bench is right**: replaying the same plants with slow detector-zero wander reverses the order (11: 7.45 → 10.07 ns; 13: 1.22 → 7.23), and on real hardware 29–30.08 the same board measured 2.96 ns (11) against 5.2–5.9 ns (13) with a flat noise floor at mid averaging times — the level of the detector's own wander, independent of loop bandwidth. The simulator's noise was too white; treat its numbers as relative, never absolute (4.6a).

4.6a How a second of the loop goes — and what the knobs move

One second, in order. *Prediction*: the state (phase, frequency, aging) rolls forward one second. Then up to two *measurements* correct it — the phase from the LTIC detector (unless railed, frozen or out of band) and the frequency from TIM2 (always; it keeps the loop alive when the detector is blind). Then the *control*: `u = -(freq + phase/KC)` — cancel the estimated frequency error and null the estimated phase over the **controller's** horizon `KC` (in force from the loop's first settle; before that, during acquisition, at the gentler `KT` pace), clamped to the detector band, sub-LSB remainder carried to the next second. The correction may grow only gradually (its largest allowed size spread over one horizon), so a restart does not kick the oscillator, but it may shrink as fast as the filter asks — until build 65 it could not, and a pull-in from half a hertz out swung the PWM from rail to rail (see the changelog). What actually reached the pin is booked back into the frequency state. Around this core: the 4σ innovation gate, the trust test (does the phase move the way TIM2 says it must? — ignoring wobbles smaller than the counter's own resolution), four seconds of deliberate phase silence after a divider arm (ramp readings then are the rail, not a phase), the start re-referencing arm, and holdover.

The honest-R insight. The R that `KL` reports (~2.9 ns on this bench) is measured from the detector's own differences at a sixteen-second lag: the white floor (~2.5 ns) plus the growth of the slow zero wander over those 16 s. The **full** zero wander is a separate structure (~2.6 ns over ~45 s on this bench) that the filter **deliberately does not chase** — it is why algorithm 13 refuses to follow slow detector structure that algorithm 11 follows without question. Measured: with the loop quiet, dph shows a flat 5–9 ns floor from 10 s to 600 s averaging **regardless of loop bandwidth** — wide, adapted and stiff all landed on the same level, the level of the detector's own wander. Whether following that wander (11) or refusing it (13) gives the better *output* only an independent reference can answer.

`KR [ns]` (default 0 = measure). Pinning to the white floor (`KR 2.5`) makes the filter follow the detector completely: sd@600 improved ~30% in quiet hours, but the 4σ gate narrows with R, so real GPS events (tens of ns) get rejected — rejects rose from 163 to 5021 per night. Pin only to bracket an experiment; `KR 0` is the better default.

`KQ [v]` (default 0 = adapt). The adaptation used to ratchet: correlated detector error keeps the innovations larger than the covariance predicts, so **Q climbed** — 195× the seed after one night, a filter five times faster than its horizon moving the DAC 1.80 LSB/s against algorithm 11's 0.49. It is now bounded at `R/KT³` — **the filter may not run faster than `KT`** —

which cuts the DAC motion 2.4× and doubles short-tau ADEV (4.6). Dan Wiering's board, same firmware, nothing touched, went the whole way to the old rail: `Q` 1000× its seed within 2h37m, the DAC moving 11.05 LSB/s, and ADEV at 20 s of 8.6e-11 against 3.1e-12 for algorithm 11 on the same board and the same rubidium. `KL` prints `[at ceiling]` when the adaptation is against the bound. Pinning at the seed (`KQ 0.000006359` — read the exact seed from the `KAL: from CT/LC` line) stiffens the loop further and nearly freezes the PWM; a value you pin passes as given, the bound is on the adaptation. **KQ saves when you type it** — put it back with `KQ 0`.

`KT [s]` (default 100; 200 in build 60). Since build 36 this is the **estimator's** horizon — the patience of the understanding, not the speed of the hands. A shorter KT means a wider belief bandwidth (and a higher Q ceiling, since the bound is `R/KT³`); a longer one leans on the oscillator's own stability. It is also the stall-check patience — five horizons far out re-arms the picDIV. The phase-nulling pace is `KC`'s job now. Which side of 100 s is right depends on the oscillator: the author's walks fast and wants 100 s; a quiet OCXO, measured against a rubidium, can want 300–1000 s, and the `KT` command's note above 200 s says how to choose: by ADEV, a run at each value — not by Q at its ceiling in `KL` (build 71; up to build 70 the note said to shorten `KT` then, and on Dan's board `KT 1000` sat at the ceiling and beat `KT 500`). Since build 67 the pull-in uses `KT` only up to 100 s — the controller's horizon, the first guess at Q, the prior on the frequency, the arm gate — so a long `KT` no longer costs a cold start: in the model of a quiet board `KT 1000` locked in 28–41 minutes before and in 6–12 now, and `KT 100` exactly as before.

`KC [s]` (default 0 = auto = KT/3, 33 s at the default `KT` — KT/6 in builds 61–67, KT in build 60, KT/3 before; **in force only from the loop's first settle** — during acquisition and after an algorithm restart the control falls back to KT, at most 100 s since build 67). How fast the *hands* null a phase error the *understanding* already has: the estimate is already smoothed, so nulling it quickly amplifies the correction, not the noise. Measured on a 20-hour log: output ADEV ~23% better at tau 256–4096, with DAC dither rising from 0.37 to 0.52 LSB/s — a budget, not a free lunch. **Judge it by the oscillator, not by the logged phase.** The logged phase is the detector's reading, and quick hands improve it by copying the detector's own slow wander into the oscillator — which is what leaves the board. What the oscillator does is the PWM integrated: its steps times the hertz per LSB, whose ADEV matched Dan's measurement of his board against a rubidium (14.10) within a few per cent from 4 to 400 s. On the author's board, in one session, `KC 16` for a night and then `KC 33` for two nights and a day: the PWM moved 664–744 times an hour instead of 1349, and that ADEV was 1.7 times lower at 4–16 s, a fifth lower at 64 s and the same at 256 s, for a fifth more logged phase (2.14–2.30 ns sd against 1.83) — build 68's default. KT/6 (builds 61–67) had been chosen on the logged phase alone: a night at `KC 16` against algorithm 11 the night before, 2.63 against 2.73 ns, with the PWM changing about 2.5 times as often, in 1–4 LSB steps. Build 60's `KT 200` / `KC` = `KT` logged 5.62 ns: it was chosen on a replay without the oscillator's own fast noise, and a controller that slow lets that noise through. In the replay calibrated against the board, `KC 25` gives about algorithm 11's phase at 1.4 times its DAC motion, and below ~15 s the phase gains little while the DAC moves much more. (The earlier sweep that found long `KC` unstable ran on a simulator that misread the fine DAC value — see the build 60 changelog.) A board keeps what it saved, and typing any of `KR`, `KQ`, `KT`, `KC` or `KF` saves all five — a board where one was typed under build 60 comes up at `KT 200`; `KT 100` + `KC 0` restore the default. `KC` with no argument prints the configured value and the one actually in force.

`KF [s]` (build 69; default 16 s, 10–250 s, `KF 0` = off). The horizon the controller uses while a **disturbance** lasts. `KC` is a choice about noise; a step of the oscillator's frequency — a supply sag, a knock, a jump of the crystal — is not noise, and a slow controller lets it push the phase that much further. On 21.10 a three-minute sag of the author's 5 V rail (to 4.37 V) moved the OCXO by $2.7 \cdot 10^{-9}$: at `KC 33` the phase went to +75 ns, swung to −51 ns when the rail came back and was inside ±10 ns only after 12 minutes; replayed with `KF 16`, +41/−29 ns and 5½ minutes. The mode goes on when the filter's phase estimate passes 10 times the board's calm noise (20 ns on the author's board, about 37 ns on a detector as noisy as Dan's) or when the gate gives way after ten refused readings, and off after two minutes in a row under 3 times that noise. It acts on the estimate, not the reading, never makes the controller slower than `KC`, and after a start or a picDIV arm of its own waits until the phase has settled (an `AP` typed at the console counts as a disturbance). The log says when it moves the horizon (`KAL: disturbance (|phase| 43 ns) - controller 33 -> 16 s`, `KAL: calm for 120 s - controller 16 -> 33 s`), the `Learn:` line ends in `KF` while it is on, and `KL` shows it. Calm operation never sees it. Measured on the author's board (26–27.10, `SP` steps of +82 and +90 LSB): with `KF 16` the phase peaked at −58 and −62 ns and was inside ±10 ns after 4.4 and 4.0 minutes; with `KF 0`, −86 and −96 ns, first inside ±10 ns after 13 and 16 minutes, then +14 and +12 ns the other way, inside for good after 25–27 and 32 minutes. Each time the gate gave way 18–26 s after the step.

`KL` — **read the state** before, during and after any experiment: estimates, phase-belief sigma, the R and Q actually in use (`[at ceiling]` beside Q means the adaptation has run into its bound of `R/KT³` — on a site with nonstop GPS micro-events that is most of the night, a property of the sky rather than a fault), last innovation, reject count, the **adaptation ratio** (innovations larger than predicted, >1 presses Q up, <1 down — labelled `Q pushed up`, `Q pushed down` or `consistent` since build 71; up to build 70 `wants more Q` and `wants less`, which read like advice on `KT` and are not) and the **Q water marks** since tracking started (`lo..hi` — did Q ever leave the ceiling overnight). The most informative habit: `KL`, then `SW`, then a one-hour log.

Trends: `KAL` (normal), `REJ` (gate rejected a reading — singles and pairs at GPS events are normal), `KEPT` (the detector repeated its last reading and the filter took no phase from it — about one second in a thousand at lock, a third of them during a pull-in; build 66), `ARM` (divider re-armed — start, railed or stalled), `NOPH` (no phase, steering on the model; **four seconds of NOPH right after `ARM` are deliberate silence** — the ramp then reads the rail, and that is not a phase), `NoPL`/`NoCT` (calibration missing), `WAIT` (no data yet). What a good night looks like — see Appendix D.

Part 5 — Displays: what every field means

Several displays can run at once (I2C + SPI don't conflict). All show the same truth, at different levels of detail.

5.1 The TFT (480x320 or 320x240)

```
y=  0..23   header bar: "GPSDO v1.08"   CPU 58%   LMT 14:32:45 Thu

The `CPU 58%` in the header bar is the total processor load, always on (the
measurement runs in the uptime task; `TL` on the serial line adds the
per-task breakdown). It appears from the second second after boot — the
100-second averaging window has no verdict yet.
y= 30..62   FREQUENCY — big digits, colour-coded
y= 70..151  info grid, two columns
y=156..195  sensor row
y=204..239  status bar
```

The big frequency number's colour is the one-glance health check:

Colour	Meaning
green	locked (for 10/11: trend `LOCK`; for 12 also during `CORR`/`ZC` — a correcting loop is a healthy loop)
white	adjusting
orange	holdover
red	no signal / no fix

While boot procedures run, the number is replaced by orange countdowns: `Survey <s> ±<m>`, `OCXO warmup <s>`, `Tune <s>` (CT), `LTIC cal <s>` (LC), `Calibrate <s>`.

Info grid: left column — UTC time + weekday, date, uptime (counted from the GPS 1PPS, so it doesn't drift), `Algo: n <trend>`, `PWM:` code + `Vct:` control voltage. Right column — `Sat:` count + `HDOP:` (or `HDOP: TIME` after survey-in — that word is *good*, it means fixed-position time mode), `Lat:`/`Lon:` (6 decimals), `Alt:` + `qErr:` (the sawtooth value being subtracted), `INA:` supply volts and current. Sensor row: `BMP:` temperature + pressure, `AHT:` temperature + humidity, `Vph:`/`dph:` detector voltage and derived phase in ns (`ovf` = ramp outside its valid band), `Vcc:`/`Vdd:` rails.

Status bar (colour-coded background). Green means one thing only: the control loop has converged, on the same verdict that colours the frequency digits above it — the band and the digits cannot disagree.

bar	colour	what it means
`DISCIPLINED FIX OK`	green	fix good, loop locked
`ACQUIRING FIX OK`	orange	fix good, loop working but not there yet
`OCXO WARMUP`	orange	the oscillator is not yet worth judging
`CALIBRATING`	orange	`C`, `CT` or `LC` is deliberately moving the output
`HOLDOVER (manual)`	orange	frozen by `MH`
`HOLDOVER (fix lost)`	red	the fix went away, or the 1PPS was missing for 10 s (build 68), and the loop froze itself
`WAITING FOR GPS FIX`	red	nothing to discipline against

`SURVEY` is appended while a survey-in is running. Green is deliberately slow to arrive and quick to leave: the loop's verdict is a one-second thing that can flicker on a single count of counter wobble, so the bar wants five consecutive seconds of lock before it turns green and two of loss before it gives it up.

5.2 The OLED (128x64) — two pages, flipping every 10 s

Page A (GPS): local time, frequency, Lat/Lon/Alt+Sats, uptime, UTC+AHT temperature, PWM + trend (a blinking `H/A` at the right edge = manual/auto holdover). Page B (sensors): BMP/AHT/INA rows, Sat+HDOP, UTC. During procedures the frequency row shows `F SVIN <s>s <m>m`, `F WARMUP <s>s`, `F CAL <s>s` instead.

5.3 The 20x4 LCD

Line 0 frequency; line 1 UTC + uptime days; line 2 rotates every 10 s (coordinates / sats+HDOP / AHT / INA / BMP); line 3 PWM + Vctl + trend with blinking holdover marker.

5.4 The LED clock digits (TM1637 / HT16K33)

Local time HH:MM (colons blink on the second). All dashes = boot/no data; `oooo` = no fix; spinners = warm-up / survey / calibration.

The spinners cost next to nothing since build 67: the display task wakes about seven times a second to step them and, on those wakes, touches only the LED digits; the TFT, OLED and LCD are redrawn once per PPS as at any other time. Before, every wake redrew everything, and the `CPU` figure doubled through warm-up and survey-in (55–56 % against 27 % on the author's board).

Brightness is a compile-time setting, one per display type, both in `gpsdo_config.h`: `TM1637_BRIGHTNESS` (0-7) and `HT16K33_BRIGHTNESS` (0-15). Neither has a CLI command - they are set once for the module you fitted and the ambient light it sits in.

5.5 LEDs on the board

LED	Meaning
Yellow (PB8)	OFF = no GPS fix *and* not in holdover · steady = fix OK · slow blink (1 s) = manual holdover, with or without a fix · fast blink (200 ms) = fix or 1PPS lost, auto holdover
Blue (PC13)	**fault only** — rapid blinking means a firmware fault was caught (assert / stack overflow / heap failure), with the reason on serial. It is *not* a heartbeat; a dark blue LED is the good state.

Part 6 — Serial telemetry (the 6-line report)

Once per second (once per PPS) the firmware prints a status block, in this shape (human-readable mode, `RH`):

```
Up: 000d 02:15:33  UTC: 22/8/2026 14:32:45
Lat: 51.477928 Lon: -0.001531 Alt: 46.5m Sat:10 HDOP:TIME
Freq: 10000000.0000 Hz  10s:0.0  100s:0.02  1ks:0.000  10ks:0.0000
PWM:44653  Vctl:1.970V hit
Learn: algo=11 (LTIC-Lars) gain=auto scale=46 phase=12.3ns LOCK qErr=-8.2ns
BMP:23.4C 1013.2hPa  AHT:22.1C 45.3%rH  INA:12.05V 250mA  Vphase:3.077V dph:1390.5ns  CPU:31%
```

(The example position is the Royal Observatory, Greenwich — longitude zero by definition, and a deliberately public place. Your unit will of course print its own coordinates; if you share logs, remember the tuner's **Redact position** option.)

Line by line:

Line	Fields
1	uptime (PPS-counted, trustworthy from v1.06) + GPS UTC date/time
2	position (6 decimals), altitude (GPS altitude **plus your `A0`**), satellites, HDOP — or the word `TIME` when the timing receiver is in fixed-position mode; no fix → `GPS: no position fix yet`
3	raw counted frequency (or `---` before the first count) and the averaged error windows 10 s / 100 s / 1 ks / 10 ks — each appears only once its buffer has filled
4	16-bit PWM code, measured control voltage, trend word (`hit`, `ACQ`, `DPLL`, `LOCK`, `PLL`, `CORR`, `ZC`, `NOPH`, `NoCT`, `ARM`, `NoPL`, …) — in holdover, `[HOLDOVER]` replaces the trend; `RAILS` replaces it while the loop is going from one end of the DAC to the other with no lock in between (build 65 — see Part 10)
5	algorithm-specific learn line (see below) + `qErr` when `SAW 1` is active
6	BMP280 temperature/pressure (**raw + your `P0`**), AHT temperature/humidity, INA voltage/current, LTIC detector voltage `Vphase`, derived phase `dph` in ns (sawtooth-subtracted like the loop itself), and `CPU:` — the share of the last second the processor did not spend idle. With an LM75 found at boot (build 68), `LM75:` follows it — the temperature in °C with three decimals, `LM75:---` once the sensor stops answering. With `TL 1` a `TL:` field follows, breaking the load down per task.

The Learn line per family:

- algo 11: `gain=auto|<val> scale=<n> phase=<ns> LOCK|acq`
- algo 12: `ph=<ns> level=<n> corr=<n> arm=<n> sig=<ns> zc=<n> secs=<n>` — accumulated phase, last acting level, corrections, arms, live noise estimate, zero-crossings, seconds since start
- algo 13: `ph=<ns> f=<ps/s> sig=<ns> R=<ns> rej=<n> arm=<n>` and `HOLD=<s>` in holdover, `KF` during a disturbance (build 69) — the filter's own estimates, not this second's reading
- algo 10: `state=ACQ|DPLL|LOCK`
- algos 3–9: learned drift/slope/damping, algo 9 adds the learned tempco

Tab-delimited mode (`RD`) prints one line per second with the same data as columns (uptime, frequency windows, sats, HDOP, PWM, voltages, all sensors, raw TIC) — the format choice for logging into a spreadsheet. A frequency-average column is **left empty until its window has filled** — the first 10, 100, 1000, 10 000 and 20 000 seconds respectively — so a plot shows missing data rather than an oscillator reading zero hertz. The field count never changes, so the separators are still there and a parser counting columns is unaffected. `RP`/`RR` pause/resume. The report is non-blocking: if a host connects but doesn't read, the firmware drops report tails rather than freeze the displays (fixed in v1.06 — a full CDC queue used to wedge the display task for good).

Part 7 — Command reference (every command)

Serial, 115200, case-insensitive, Enter-terminated. Commands with a `[val]` argument **show the current value when called with no argument**. Two saving regimes exist — the firmware always tells you in its reply which one you just hit:

- **Preferences save themselves** and print `[auto-saved: …]`: `TZ`, `TO`, `LT`, `WU`, `SPL`, `SV`, `PO`, `AO` — plus `CT` (auto-saves the PID group it derives) and `LC` (auto-saves on PASS).
- **Loop parameters live in RAM until you save them** — everything the tuner edits: gains, LTIC/Lars/algo-12 parameters, `LA`, `SP`. The reply names the exact group to keep the change (`[not saved — run 'ES …']`).

Version, help, state

Command	What it does
`V`	version, authors, credits — and the **CRC-32 of the flash image**, computed at boot from the flash itself. Unlike the compile timestamp it cannot be stale: the Arduino builder reuses object files, so a sketch you have not edited keeps an older date. When a log and a memory disagree, trust the CRC.
`H` / `?`	command list · `H TZ` = timezone details
`SW`	stack watermarks, free heap, uptime + its source, MCU-vs-GPS ppm — and **CPU load per task**, busiest first, averaged over a real 100 s window of one-second buckets. Measured on the Cortex-M4 cycle counter at every context switch, so it is exact rather than sampled. Interrupt time lands on whichever task was interrupted, so read it as "the processor was here", not "this task used this".
`TL 0|1`	put the same per-task figures on the telemetry line as a `TL:` field. Off at every boot and never stored — it is a bench diagnostic, not a setting.
(none)	`RP` / `RR` pause and resume the telemetry — a one-key TAB/ESC toggle was tried and removed: real terminals and the tuner send CR/LF-terminated lines and never a bare TAB or ESC, so it could not be reached from the tools people actually use.

Output (the DAC)

Command	Range	What it does
`SP [n]`	1..65535 (no arg = 32767, mid-scale ≈1.65 V)	set the control DAC directly — manual control during experiments
`up1/up10/dp1/dp10`	—	nudge PWM ±1/±10 (refused during a running calibration)
`DAC`	`PWM/DITH/EXT`	no argument: report — output path, 24-bit and 16-bit codes, commanded **and** measured Vctl, one step in µHz (needs CT), and on a span-jumper board which position the plant belongs to. With an argument: select the active output path — **auto-saved**. The jumper on the board switches the signal; this tells the firmware which path it is steering. Unset defaults to `DITH`.
`DV [v]`	2.50..5.50 V	volts at full code, for the report's "commanded" — **auto-saved**. `3.30` (the default) is the PWM model; set `5.00` on an AD5680 board. Three decimals are kept (`4.096` for a 4.096 V reference).
`AV [x]`	1.00..10.00	divide ratio of the divider before the ADC pin, for "measured" and every Vctl display — **auto-saved**. `1.00` (the default) = direct.
`VS [VCC|VREF]`	—	what the PA0 divider is jumpered to — **auto-saved**. `VCC` (the default) is the 5 V rail, as on every board before V3. `VREF` is the voltage reference, and the `DAC` report then checks the reading against `DV` and complains past 5 %.

Command	Range	What it does
`SPAN [CLR FULL|REDUCED]`	—	the EFC span jumper on PB14 (`GPSDO_SPAN_SENSE`): no argument = the position the jumper is in and the `CT` calibration (K and null) each position carries. `CLR` forgets one position's calibration — for one measured in the wrong place. The `DAC` report prints the same lines.

Three paths, one node. All three output paths can be compiled together; the `DAC` command picks which one the firmware writes, and the board jumper decides which signal reaches the EFC filter (there is no soft multiplexer — two drivers on one node would be a hardware fault). At boot every compiled path is brought up so a switch is glitch-free, but a control step writes **only the selected path**: an unselected AD5680 holds its last code, and while `EXT` is live the PWM pin is held low (since build 69; before that it went on replaying its last code, a carrier on the PWM filter). `PWM` is the plain 16-bit PWM on PB9 (on a build with the dither engine: that engine at whole-LSB granularity — same voltage, no reconfiguration). `DITH` is the 13-bit carrier + 1-bit sigma-delta dither on PB9/TIM4 CH4, an effective 24 bits in time-average (carrier ≈11.7 kHz at the BlackPill's 96 MHz timer clock — `DAC` prints the board's own — DMA). `EXT` is the AD5680 over bit-banged SPI on PB4/PB0/PB2, written about once per second (~20 µs with interrupts masked).

Internally the control value is **one 24-bit code**; the 16-bit value on the display, in the flash ring and in the logs is that number rounded — a view, not a second variable. Fine corrections accumulate in the low byte (a 24-bit code that is not a multiple of 256 is the proof the fine path is driving; the `DAC` report marks it). On `EXT` the command is mapped onto the **18-bit** part by scaling, not truncation: `code18 = code24 × 262143 / 16777215`. The AD5680's SPI frame is 24 bits — `[0000][D17..D0][00]` — with 18 significant, so "24-bit control voltage" in the docs means the command the loop carries (and which `DITH` genuinely realises as a time-average); the part itself resolves 18 bits. The scaling keeps 16-bit full scale landing exactly on the part's full scale, which is why K and every coefficient measured on one path carry over to the other unchanged. At 5.0 V one DAC LSB is 19.1 µV; on `EXT` the reference *is* the control voltage, so its low-frequency drift maps 1:1 onto frequency.

The report's commanded-against-measured check is honest only when the two scales match the board: `DV` states what full code means at the output (3.30 V = the PWM model, the default), `AV` states the divide ratio before the ADC pin (1.00 = direct). On a 5 V external DAC with a divider at the sense point, setting both turns the check from a PWM-board assumption into a statement about this board — without them it will flag MISMATCH on every reading. Both auto-save.

Two spans, two calibrations (`GPSDO_SPAN_SENSE`, PB14). A board with the EFC level shifter can run the oscillator over the full control span or a reduced one, and the two are different plants — on the V3 prototype K is about 3.5 times smaller in the reduced position (3.3–3.6×, both measured on one reference). Wire the span jumper's second pole to **PB14** (jumper fitted = PB14 to ground = REDUCED; open or unwired = FULL) and the firmware keeps one `CT` calibration per position:

- when the jumper moves (debounced, about half a second) the loop coefficients are re-derived from that position's K and the control code is remapped so the EFC pin keeps its voltage — the frequency before the move is the frequency after it. The remap needs one pair of codes known to give the same voltage in both positions; the firmware learns it by itself — the code the loop was locked at before a move, paired with the code it locks at afterwards or with the 10 MHz code `CT` finds there. Only a code that really is a null counts: not one held in holdover, not one set by hand (`SP`, `up`/`dp`) until the loop has unlocked and locked again, and not one far from where `CT` found 10 MHz in that position — it must lie within 5e-8 of that position's null and no nearer the other position's (within 5e-9 of its own null it always counts). The last test catches a loop locked on the wrong plant, which is what a hand-wired bench produces when the shifter goes in or out before PB14 follows. Every refusal is printed. Until there is a pair, a move keeps the code and the loop re-acquires;
- a position that has never been calibrated runs on the other position's coefficients — as a one-calibration board always did — while `CT` starts by itself as soon as there is a GPS fix (never in holdover), with up to three more attempts 10, 20 and 40 minutes after a failure;
- a jumper moved while the board was off is handled at power-on, before the loop starts. From build 65 the stored code is remapped from the span the live store's slot says it was written in, not from the span the settings record names — the two are separate writes, and a power cut between them after a move, or a record whose save failed, used to start the oscillator over 1e-7 off.

`SPAN` shows the state, and the `DAC` report prints it under the plant line. A board calibrated before PB14 was wired has its one calibration filed under the position the pin read the first time it booted with span sensing — FULL, if the wire was not there yet. If that calibration was really made on REDUCED, the first `CT` in the other position measures the same plant, says so, and forgets the wrong one; `SPAN CLR` does the same by hand. Manual gains in LSB (`LG`, `MG`, `LTK`) are not rescaled — a move warns about each one that is set.

`SPAN` also prints each position's **null** — the code `CT` found 10 MHz at, kept current by the pairs measured since — which is what the test above compares against. Nulls come only from `CT`, so a board calibrated before build 58 has none: run `CT` once in each position and the test is on. `SPAN CLR` forgets a position's K and null, and the pair.

Calibration

Command	Duration	What it does
`C`	~2 min	older 2-point PWM centring
`CT`	~3 min (+8 on gentle K)	**oscillator sensitivity K + auto-tunes PID 3–9 and LTIC; auto-saves PID** — run this first. On a gentle plant (small Hz/LSB — span-reduced EFC, external DAC) a second pass re-measures around the fitted 10 MHz code: 0.8 Hz of swing whatever the span, each point a 100 s average, so K comes out to about 1 %
`LC`	~5 min	**phase-detector calibration (ns/V, offset, range); auto-saves on PASS** — run this second
`ACG g [cap]`	—	ACQ centring drive: gain 50..20000 LSB/V, max step 5..1000 LSB
`AP`	—	arm the picDIV divider manually

Mode and reporting

Command	What it does
`RH` / `RD`	human-readable / tab-delimited report
`RP` / `RR`	pause / resume the 1 Hz report
`TL 0|1`	per-task CPU load on the telemetry line (`SW` shows it once; the TFT header shows the total CPU% always)
`MH` / `MD`	holdover (freeze PWM, fly alone) / disciplined
`F`	flush the frequency averaging ring buffers
`T [baud]`	transparent GPS tunnel on USB for u-center, 300 s (baud 4800..921600)

Algorithm selection and classic PID (see Part 4)

Command	Range	What it does
`LA [n]`	0..13	select / show the loop algorithm
`LP [n]`	algo 0..9	list PID parameters
`KP/KI/KD n val`	algo 3..7, 0..100000	set gains
`IL n val`	algo 3..9, 100..100000	set integrator clamp
`BC` / `BS`	0.0001..1.0 Hz	algo 8 blend crossover / scale
`NS`	1..10000 LSB	algo 9 max step

LTIC (algo 10) — save with `ES LTIC`, list with `LL`

Command	Range / de-fault	Meaning
`LNV`	0..1e6	detector slope, ns per volt (LC measures it)
`LZO`	0..3.3 V	detector zero-offset volts
`LRN`	2..1e9 ns, or `0`/`1`/`R`	detector range, ns (`LC` measures it; decimals and exponents are fine — `LRN 2500.7`, `LRN 1e3`). The same verb controls self-learning — the drift feed-forward and damping of algorithms 3–10: exactly `LRN 0` freezes it (what it has learned stays applied), `LRN 1` resumes it (default on), `LRN R` returns drift and damping to theory and saves that at once, bare `LRN` shows it. Anything else is refused. `LRN 0`/`LRN 1` are saved with `ES FLAGS`
`LAT`	0.001.. 1e9 ns (100)	ACQ phase threshold
`LDT`	1e-13..1.0 (5e-10)	DPLL→LOCK drift threshold
`LIV`	1..600 s (300)	LOCK correction interval
`AQP/AQI/AQD/AQL`, `DPP/DPI/DPD/DPL`, `LKP/LKI/LKD/LKL`	0..100 000	stage PIDs
`LPOL`	−1 / 0 / +1	PWM→phase polarity (0 = not set: the loop holds and says so; run `LC` to measure it)
`LCV`	0..3.3 V	ACQ centring target
`LRM`	0 / 1 (1), not saved	how the detector burst is read: 1 = a line through 64 conversions, read at the edge itself; 0 = the median of 16, as up to build 60
`LB`	—	counts since boot (edge caught, line refused and why, reading kept, a third left out and the typical bend moved — build 71), then the last burst: taken at the edge or late, why its line was refused, each conversion's time and code, the line, its spread, its bend and the board's typical one, how far its thirds are from each other, how fast it falls
`FA/FAD/FAL`	10 / 100 / 1000 s	damping average window (both / DPLL / LOCK)

LTIC-Lars (algo 11) — save with `ES LTIC`

`LG` (0..10000, 0=auto), `LD` (default 3), `LTC` (1..600 s, default 60), `LFD` (1..100, default 2), `LTO` (0..3.300 V, default 2.111 V), `LPL` (1..10000 ns, default 100), `LPF` (1..100, default 5), `LTK` (±32000, 0=off), `LTR` (0..3.300 V) — meanings in Part 4.4.

Algo 12 (multi-level) — save with `ES ALGO12`, list with `ML`

`MG` (0..10000 LSB/ns, 0=auto from CT), `MR` (0..10, default 7), `MM` (0..1, default 1 = calm), `MF` (0..3 limits source, default 0), `MFT` (0=3600 s, or 2048..65535 s), `MLP <level> <ns>` (level 0..10) — meanings in Part 4.5.

Algo 13 (Kalman) — saved on entry, no `ES` needed

`KR` (0..1000 ns, 0 = measure), `KQ` (0..1, 0 = adapt), `KT` (10..10000 s, default 100), `KC` (10..10000 s, 0 = auto = KT/3; in force after the first settle), `KF` (10..250 s, 0 = off, default 16: the horizon while a disturbance lasts), `KL` (list the filter state) — meanings in Part 4.6.

GPS, time, sensors

Command	Range	What it does
`SV 0|1`	—	survey-in / Time Mode off/on (applies at next boot) — **auto-saved**
`TZ <city|rule>`	e.g. `TZ Adelaide`	timezone with DST (`H TZ` for details) — **auto-saved**
`TO <h[:mm]|A>`	−14..+14	fixed UTC offset, or `A` = auto from GPS position (EU DST rule) — **auto-saved**
`LT 0|1`	—	show UTC / local time — **auto-saved**
`PO <f>`	−5000..5000 Pa	pressure offset added to the BMP280 reading — **auto-saved**
`AO <f>`	−3000..3000 m	altitude offset added to the **GPS altitude** — **auto-saved**
`SAW 0|1`	—	sawtooth (qErr) correction off/on — bare `SAW` shows status; recommended ON with a timing receiver (save with `ES FLAGS`)
`WU 0|1`	—	OCXO warm-up on boot — **auto-saved**
`SPL 0|1`	—	boot animation on/off — **auto-saved**
`BL [%]`	30..100	TFT backlight brightness — **auto-saved**. Bare `BL` reports it.

The backlight, and why 100 % is the quiet setting. `BL` drives a P-channel MOSFET on **PB5** (TIM3 CH2, 20 kHz) between the 3.3 V rail and the panel's LED anode: gate through ~47 Ω, a 100 kΩ pull-down to ground, and 10–47 µF of bulk at the drain. The stage **inverts** — a low gate is full brightness — so the firmware writes the complement, and `BL 100` lands on a compare of zero: the pin sits statically low and the stage does not switch at all. That matters because it is the 3.3 V rail that feeds VDDA, and therefore the ADC reference, the phase reading and the Vctl monitor. Full brightness is the quietest setting, not the loudest; dimming costs a little rail noise and buys back load and heat in an enclosure that is thermally coupled to the OCXO.

The floor is 30 % for a reason that is not electrical: below about a third the panel stops being readable rather than becoming usefully dim, so a mistyped `3` would leave you looking at what could equally be a dead board. On a board without the MOSFET the pin simply carries a square wave and nothing happens. `BL` is compiled in with any TFT, and **yields PB5 to the 2 kHz test generator** if that has been explicitly enabled — a default must not take a pin from a deliberate choice.

Saving, recalling, resetting

Command	What it does
`ES [obj]`	**save** everything (no argument) or one group: `TZ`, `PID`, `LTIC`, `FLAGS`, `ALG012`, `ALG0`, `PO`
`ER`	**recall** — re-read settings from flash now (undo unsaved changes)
`EE`	**erase** the settings slot — defaults on next boot
`EW`	flash-ring wear: erase cycles, slots used, sector/address
`FR`	read-only ring status (always on since v0.96)
`CS`	correction statistics: count, peak, RMS over the last 100/1k/10k/100k corrections — small and steady is good, growing is bad

Command	What it does
`RB`	warm reboot (keeps settings — but does **not** auto-save first)
`CR YES`	cold restart: wipe settings + learned data, factory defaults (the YES is required because the learned model takes days to rebuild)

Known quirks in v1.08 (honest list)

None known at the time of writing. The two listed for v1.07 — `LRN` reading its argument as a whole number (`LRN 1e3` switched learning on, `LRN ON` switched it off) and `LRN R` suggesting an `ES FLAGS` that does not save the reset — are fixed in build 58.

Part 8 — The PC tuner

`tools/gpsdo_tuner.py` — a desktop console for watching and tuning. It does everything a terminal does, plus live plots and CSV logging.

8.1 Install and start

Python 3.9 or newer, then:

```
pip install PySide6 pyqtgraph pyserial tzdata
```

(`tzdata` is only used by the *Generate tz_table.h* button; on Linux/macOS the system provides it.)

Run: `python gpsdo_tuner.py` (on Windows you can double-click it).

8.2 Connect

Pick the port in the toolbar, baud 115200, Connect. The tuner immediately reads the firmware version and **all** current parameters (`LL`, `FA`, `LP 3`–`LP 9`, every Lars and algo-12 verb, `ML`) and fills every tab — you see the device's real state, not defaults. The state line shows `state: LOCK (locked)` etc.; plot titles follow the active algorithm.

8.3 The tabs

Tab	What it edits
LTIC (algo 10)	the three stage-PID quartets; Read (`LL`) / Save (`ES LTIC`) / Revert (`ER`)
LTIC-Lars (algo 11)	`LG LD LTC LFD LTO LPL LPF LTK LTR` with Set buttons
Multi-level (algo 12)	`MG MR MF MFT` + the whole 11-row limit table (`Send limits`, `Read all`, `List (ML)`, `Save (ES ALGO12)`)
FA damping	DPLL / LOCK damping windows
PID algo 3-9	Kp/Ki/Kd/IL per algorithm
Calibration	`LNV LZO LRN LCV LAT LIV LPOL`
Raw monitor	everything the board says, unparsed; logging controls
Help	the firmware command reference

Remember the firmware rule applies here too: **Set buttons change RAM only; the tab's Save button runs the `ES ...` that persists.**

8.4 The plots

Three panes, refreshed continuously from the 1 Hz telemetry. Top panes depend on the algorithm family — phase (`dph`/`ph`) + detector voltage for 10/11/12 (with grey anchor and red valid-band guide lines once calibration is known), drift + control voltage for the classics. The bottom pane is always the frequency error in Hz. Toolbar: time-span selector (1 min … all), Follow live, Clear. Resolution is the telemetry rate — events faster than ~2 s are invisible, and the plots trust the board.

8.5 Logging (the plots are not a logger)

Raw monitor → **Start logging**, format Full log / CSV only / Both:

- **Full log** `gpsdo_YYYY-MM-DD_HH-MM-SS.log` — every line as printed, ~217 MB per week at 1 Hz.
- **CSV** — parsed columns: `utc, up_s, algo, state, dph_ns, qerr_ns, vphase_v, pwm, f10, f100, ph_ns, level, corr, sig_ns, zc, bmp_c, sat, hdop` (~65 MB/week). Empty cell = field absent that second (never zero).
- **Redact position** (default ON) blanks Lat/Lon/Alt in the saved full log — share logs without publishing your rooftop.

Part 9 — Settings, the flash ring, and wear

Everything persistent lives in one **wear-levelled ring** in flash sector 7: 255 slots of 512 bytes. Each save (an `ES`, a successful `LC`, or a learned-data update) writes the next blank slot; the sector is erased only when the ring wraps — once per 255 saves — and a wrap carries the newest settings, Kalman and live records across the erase (from build 65; until then it kept only the slot being written, and the next boot could come up on defaults). At the rate this firmware saves (even a busy bench hits ~73 saves/day) that is an erase every ~3.5 days, and the F411's flash survives ~10 000 erases per sector: **roughly 96 years**. You will never wear it out. `EW` shows the actual counts any time you're curious.

Two kinds of records share the ring:

- **Settings** (`ES` and friends) — everything you chose, saved only when you say so. Partial saves (`ES PID` etc.) write the *whole* settings block seeded from the last stored one, so saving one group can't clobber another.
- **Live data** — what the device learns by itself: the `LC` calibration, the learned drift/damping model, the last operating point. Saved automatically, but with hysteresis (only when something actually changed by a meaningful amount, and at most every 20 minutes); `LC`, `LRN R` and a span move save it at once. The operating point is saved on its own (from build 65) when it has moved more than 0.02 Hz from the saved one (in LSB through `CT`'s K; 64 LSB without a `CT`) and the loop has been locked for 20 minutes without a break and without a jump (a code typed with `SP`, `up` or `dp`, however small, or left by a calibration starts the 20 minutes again) — never in holdover, during a calibration or while a span move is being settled; algorithms 0–9 have no lock and never save it this way. Up to build 64 it was written only with the drift model, which algorithms 11–13 do not move, so a cold start could begin half a hertz off. The slot also says which span its code belongs to; a slot written by build 64 still loads. A save the flash refuses is tried again 20 minutes later.

Firmware upgrades: settings written by v1.04/v1.05 are migrated on recall (v4→v5 adds the algo-12 block at its defaults); a settings block from a radically different version, or a corrupt slot, is treated as "not there" — defaults, and the firmware asks for a calibration on the next boot. The ring itself self-heals: a slot half-written by a power cut fails its CRC and is simply skipped; the previous good slot wins.

The practical checklist, one last time:

- routine re-flash: safe, settings survive (Part 2),
- after a tuning session: `ES`,
- before selling/gifting a unit: `CR YES`,
- when a unit behaves strangely after an experiment session: `ER` (reload the saved, known-good settings), and only then start diagnosing.

Part 10 — Troubleshooting

Symptom	Likely cause → fix
Won't compile, `ltoa` error	STM32 core 3.0.0 — downgrade to 2.12.0 (Part 1.1)
Compiles, prints `?` or garbage where numbers belong	Tools → C Runtime Library → **Newlib Nano + Float printf/scanf**
TFT white after core 3.0.0 build	same — core 2.12.0
TFT white with 2.x core	`User_Setup.h` wrong driver or `TFT_MISO PA6` missing (Part 1.4)
Boot freezes right after `TFT: init start`	same as above — check wiring and the driver `#define`
USB serial missing after upload	press RESET; if still gone, Zadig driver for VID 0483/PID 5740 (Part 2.5)
Settings/calibration gone after flashing	someone used **Erase Chip** — that's the sector 7 rule (Part 2.2); re-run `CT`, `LC`, `ES`
Yellow LED off	no GPS fix and not in holdover — antenna, sky view, cable. In holdover it always blinks, so a dark LED never means "frozen output"
`HDOP:TIME` never appears	survey-in not finished (needs timing receiver + `SV 1` + good sky); it re-runs after each power loss
`CT` fails / rejects K	EFC wiring or oscillator sensitivity out of 0.01–2 mHz/LSB range — check the DAC buffer and EFC span
`LC` fails	run `CT` first; keep the unit still during the 5 minutes
Trend stuck `NoCT` (algo 12)	gain is auto but CT never ran → `CT`
Trend stuck `ACQ` / `NoPL`, algos 10–13	`LPOL` unset (the loop prints a warning and holds) — set `LPOL -1` or `1`, then `ES LTIC`
Algo 12 corrects constantly at level 0	noisy site — `MF 1` (Alan's stored table), see Part 4.5
Phase ramps away after `SAW 1`	you don't have a timing receiver / qErr invalid — `SAW 0`
Report lines freeze when a program opens CDC but doesn't read	fixed in v1.06 (non-blocking writes + 1 KB queue); if you see it, you're on old firmware
Blue LED blinking rapidly	firmware fault caught (assert / stack overflow / heap) — read the serial message; report it
Unit resets spontaneously	check the 3V3 rail under OCXO load; the boot banner prints the reset cause
Display alive, no serial output	you're on Bluetooth only? 57600 on Serial2; or `RP` was typed — `RR` resumes

Symptom	Likely cause → fix
`LOOP: rail to rail with no lock …` on the console, trend `RAILS`	the loop took the PWM from one end of the DAC to the other without locking in between: it is diverging, not pulling in (build 64's algorithm 13 did this from half a hertz out; fixed in 65). `LA 11` pulls in from far off; if algorithm 11 itself does it, `CT` (K) or `LPOL` is wrong. Without the detector (no `GPSDO_LTIC`) the line ends "Check CT." instead. The warning clears on lock, on a change of algorithm, when something other than the loop sets the code (`SP`, `CT`, `LC`) or after 30 minutes without either end

If none of that helps: capture a full log with the tuner (Part 8.5), note the `V` version string, and ask — with the log attached. A week of 1 Hz CSV is the difference between guessing and knowing.

10.1 How to report a problem

Nearly every question about this firmware has been answered by four things, and answering it without them takes days of guessing instead of minutes of reading. Send all four:

1. **The boot log** — everything from reset down to `GPS init done`, copied as text rather than photographed. It names the board, the detected GPS baud rate, which sensors answered, which UBX frames were ACKed and what the reset cause was.
2. **Your compiled `gpsdo_config.h`** — or just the list of switches you changed. Most surprises are configuration differences, not faults: a board without the phase detector, a second display on the same pins, `SAW 1` on a navigation receiver.
3. **Which GNSS module** — a genuine u-blox timing receiver, a genuine navigation module, or a clone (see Part 3.2). Clones behave differently and knowing which you have removes half the possibilities immediately.
4. **The antenna and where it sees sky.** "Indoors on a window sill" is a perfectly good answer and often the whole explanation.

One check before you send: the banner must contain a `compiled <date> <time>` line. If it does not, you are running a build from before v1.05 and the first thing to try is a current one — several reports have turned out to be bugs that were already fixed. The line is printed by the firmware itself, so it cannot be out of date with the binary the way a remembered version number can.

If the board **freezes**, add one observation that costs nothing: is the blue LED on PC13 still blinking? It is toggled in the 2 Hz timer interrupt and owes nothing to any task, so blinking means the MCU is alive and a task is stuck — a completely different fault from a board that is not running at all.

Appendix A — How a GPSDO works, in plain words

You can use this device without reading this appendix — but the day something behaves oddly, this is the twenty paragraphs that will tell you *why*.

A.1 The problem

You want a 10 MHz signal that is right **now** (second to second) and right **forever** (over months and years). Two building blocks each deliver half of that, and neither delivers both:

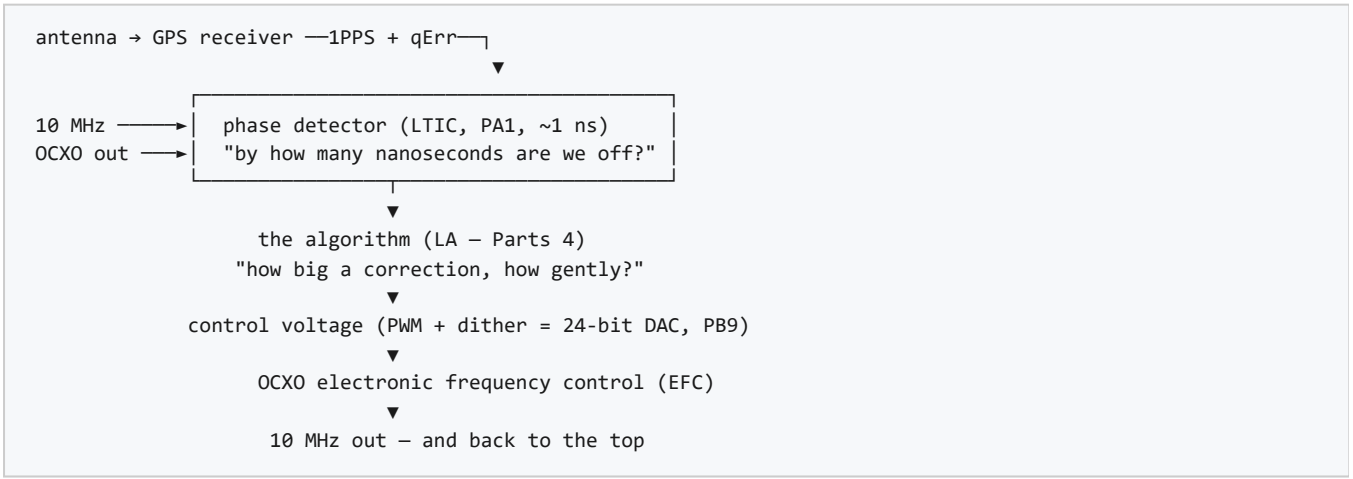
	Short term (seconds)	Long term (months)
OCXO alone	excellent — a good one is steady to parts in 10^{11}	drifts — aging and temperature pull it slowly off frequency
GPS 1PPS alone	noisy — each pulse lands within ~±25 ns of truth	excellent — it is steered by atomic clocks, forever

The oven part of "OCXO" is half the story already: the crystal sits in a little heated box held at one constant temperature (that's the 300-second warm-up at boot), because temperature is the crystal's biggest enemy. What remains is aging — a slow walk nobody can switch off.

The GPS pulse is the opposite: every individual second is only approximate (the receiver quantises time, the signal slows in the ionosphere, bounces off buildings), but the *average* over hours is nailed to atomic time, because the satellites carry atomic clocks and GPS cannot work otherwise.

A GPSDO is a division of labour: the OCXO handles the seconds, GPS handles the months, and a control loop in between moves the first, in tiny steps, to keep it anchored to the second.

A.2 The loop, piece by piece — and where each piece is on this board



- **The phase detector answers "by how much are we late?"** once a second, in nanoseconds. That single number — the *phase* — is the scoreboard the whole game is played on.
- **The counter (TIM2) answers "are we fast or slow?"** by counting the actual 10 MHz. It's coarse (0.01 Hz steps) but direct.
- **The algorithm** looks at those numbers and decides one thing: how much to nudge the control voltage, and over what time to spread the nudge.
- **The DAC**: 65536-step PWM, dithered to an effective 24 bits (steps of ~0.2 µV — the oscillator notices far less than one PWM step, so the firmware remembers the fraction between steps).
- **The EFC pin of the OCXO**: volts in, frequency out — the whole adjustment range is only a few hertz wide, which is exactly why microvolts matter.

A.3 Phase, frequency, and the one multiplication you need

Frequency error and phase drift are the same fact in two costumes. If the oscillator is off by a fraction, its phase slips at exactly that rate:

Frequency error at 10 MHz	Phase slips
1 Hz	100 ns every second
0.01 Hz	1 ns every second
0.001 Hz (1 mHz)	1 ns every 10 seconds

Two consequences worth engraving:

1. **If the phase stops moving, the frequencies are equal** — whatever the phase value is. The loop's real goal is a *still* phase, parked near zero.
2. The errors worth chasing are absurdly small. One millihertz at 10 MHz is one part in 10^{10} — and the loop routinely resolves ten times better than that. This is why every wire, every microvolt and every degree matters more than in any other circuit you have built.

A.4 Why corrections must be gentle (the heart of the whole subject)

Suppose the GPS pulse arrives 20 ns late because the signal bounced off a building. The phase "error" you measure is 20 ns — but it isn't real: the oscillator didn't move. If the loop corrects immediately at face value, it **copies the GPS noise onto the oscillator** and the output ends up *worse* than the free-running OCXO. That single mistake is what separates a GPSDO from a GPS-flailing oscillator.

The defense is averaging: random noise shrinks by √N. Average 100 seconds → noise ÷10; average 1000 s → ÷31. So the loop waits until the averaged GPS numbers are quieter than the oscillator's own drift, and only then acts — with a correction sized to what the long average actually proved, spread over a comparable time. That is why:

- algorithm 11 has `LTC` (a time constant — "how patient is this loop"),
- algorithm 12 lets **the size of the error choose its own averaging time** (a big error is proven in seconds and acted on in seconds; a tiny one is proven over an hour and corrected over an hour),
- every correction in this firmware is dithered into sub-LSB steps rather than dumped at once.

One free lunch: **sawtooth correction (`SAW 1`)**. The receiver *knows* how far it quantised each pulse (the `qErr` number) and confesses it every second. Subtracting the confession turns ±25 ns of fake error into a few ns before any averaging even starts.

A.5 Holdover

When GPS disappears (antenna cut, receiver dead), the loop has nothing true to steer by. The right move is to **freeze** the last good control voltage and let the OCXO coast on its own stability — that's holdover (`MH` manual, or automatic when the fix goes or the 1PPS has been missing for 10 s; the yellow LED blinks, the display goes orange, `[HOLDOVER]` replaces the trend in the report). The oscillator then drifts with aging and temperature — slowly, but unstoppably, until GPS returns. The pulse is checked as well as the fix since build 68, because a timing receiver in Time Mode reports its fixed position as valid with no signal at all; and automatic holdover ends only when the fix is back with 10 s of unbroken pulses, so an antenna that comes and goes does not end it with a pulse or two. A manual holdover stays manual through such a loss (build 68): the loss is reported (`GPS fix lost – manual holdover kept`, or `GPS 1PPS lost – …`), and the holdover is yours to end with `MD`, not the GPS's.

A.6 How results are judged: Allan deviation in one paragraph

People compare frequency standards with **Allan deviation (ADEV)**: roughly "how much does the average disagree with itself, taken over τ seconds", plotted against τ. A GPSDO plot falls from left to right: at τ = 1 s you see the oscillator's own noise; as τ grows, averaging beats the noise down and the GPS anchoring takes over (on the author's bench: ~$3{\cdot}10^{-9}$ at 1 s falling to ~$5{\cdot}10^{-12}$ at 3000 s). When someone posts "1e-12 at tau 10 000", that is the sentence they are say-

ing. A *hump* in the middle of the curve means the loop is fighting itself — too fast for its own noise — and is the classic signature of a badly tuned GPSDO.

Appendix B — PID for the reluctant

You never *need* this appendix to use the device — `CT` tunes the loops for you. This is for the day you open the **PID algo 3-9** tab in the tuner, or type `KP`, and want to know which lever you are holding and which way it bites. No mathematics beyond multiplication is used.

B.1 The shower picture

Every control loop ever built does the same four things:

1. **Measure** where you are (water temperature).
2. **Compare** with where you want to be (comfortable).
3. The difference is the **error** (too cold by 5 degrees).
4. **Act** (open the hot tap), then wait and repeat.

The only question in the whole subject is step 4: *how much do you open the tap?* PID — three letters, three answers — is the standard recipe.

B.2 P — proportional: "react to now"

Open the tap in proportion to the error. Cold by a lot → open a lot. Cold by a little → open a little.

- **P alone has a flaw:** it needs some error to produce any action at all, so it settles *near* the target, never at it — the last degree of cold isn't enough to hold the tap open. That residual is called **droop**.
- **Too much P:** you overshoot hot, then overcorrect cold, and oscillate between the two — the shower-from-hell.
- **Too little P:** takes forever to get anywhere.

B.3 I — integral: "remember the past"

Watch the error *over time*. Even a tiny persistent coldness accumulates, in a running sum, until the loop adds enough action to close it. **I is what kills the droop** — it is the "eventually, exactly right" term.

- **Too much I:** the classic slow yo-yo. The sum builds up during the approach, then spends itself as overshoot, rebuilds the other way, and the loop swings gently for ages.
- **Windup:** if the tap is already fully open (the output is at a limit) the sum keeps growing uselessly and then takes forever to unwind. The fix is a clamp on the sum — that is exactly what `IL` (I_LIMIT) is.

B.4 D — derivative: "anticipate"

React not to the error but to how *fast* it is changing. Warming quickly? Start closing the tap *before* you arrive. **D is the damper** — the suspension that stops the oscillations P and I would otherwise love.

- **D's vice:** it amplifies measurement noise (differentiating a jittery sensor produces spikes). That's why D terms are usually filtered, small, or both — and why a noisy phase measurement makes D-heavy tuning worse, not better.
- In the PLL algorithms (4/5/7) the `Kd` slot acts on the accumulated *phase* rather than a raw derivative — same damping duty, different wiring. Don't let the letter confuse you; think "the damping knob".

B.5 The three letters, one table

Knob	Reacts to	Cures	Too much →	Too little →
P	error right now	sluggishness	overshoot, fast oscillation	takes forever
I	error accumulated over time	standing offset (droop)	slow yo-yo oscillation, windup	settles near, never at, the target
D	how fast error changes	overshoot (damping)	nervous jitter chasing noise	overshoot rings

B.6 Where the knobs live in this firmware

You are turning	Where	Which letter it really is
`KP n val` / `KI` / `KD` / `IL n val`	algorithms 3–9 (tuner tab **PID algo 3-9**)	literally P, I, D and the integrator clamp
`AQP…AQL`, `DPP…DPL`, `LKP…LKL`	algorithm 10's three stages (tab **LTIC**)	a full PID set per stage — the loop re-tunes itself as it settles
`LG` / `LD` / `LTC`	algorithm 11 (tab **LTIC-Lars**)	gain (how hard), damping (how it settles), time constant (how patient) — the same three knobs wearing Lars' clothes
`MG` and the limits	algorithm 12 (tab **Multi-level**)	no PID at all — see below

Algorithm 12 deserves one honest paragraph: it has **no P, I or D**. Instead of fixed gains it asks, each time, "how big is the error, how long did I average to prove it?" and scales the correction to match — big errors get fast, firm treatment; small ones get slow, gentle treatment. It is the same physics with the tuning built in, which is why its only hand-knobs are the gain (`MG`, and `CT` measures that for you) and the limits that decide what counts as "big".

B.7 Why `CT` exists — the ruler problem

PID numbers live in units of "PWM steps per hertz of error". But one PWM step is worth a *different number of hertz on every individual oscillator* — 0.32 mHz on one of the author's Vectrons, nearly seven times less on a narrow-EFC build. Without measuring your oscillator, the same `Kp = 1000` is a gentle nudge on one board and a violent shove on another. **`CT` measures your oscillator's volts-to-hertz response and rescales every gain to match**, so the shipped tuning means the same physical thing on every unit. This is also why the manual's calibration order is law: `CT` first, `LC` second, tuning (if ever) last.

B.8 Ten rules of thumb

1. Run `CT` and `LC` before touching any gain. In most lives, that's the end of tuning.
2. Change **one** knob at a time.
3. Halve or double — never ×10. Loop tuning reacts to ratios, not arithmetic.
4. Give every change an hour. These loops average over minutes; judging a change after thirty seconds is reading a book by one letter.
5. When in doubt, go **slower** (bigger `LTC`, smaller gains). Slow is merely unexciting; fast is unstable.
6. Oscillation → cut P first, then I.
7. An offset that never quite closes → more I — or, more likely, you skipped `CT`.
8. Output visibly chasing GPS noise → slower loop, `SAW 1`, better antenna view. Not bigger damping.
9. `ES` after every session; `ER` un-experiments a bad afternoon.
10. If it fights you for a whole day, suspect hardware before tuning: antenna sky view, temperature (heaters, doors, sunlight), EFC wiring. The loops in this firmware are hard to break and easy to blame.

Appendix C — Glossary

Terms that appear throughout this manual, the telemetry and the changelog, in the sense this project uses them. Suggested by Alan Cashin, who pointed out that half of them are used differently elsewhere.

Term	What it means here
ADEV	Allan deviation — the standard measure of oscillator stability: how much the fractional frequency changes between adjacent averaging windows of length τ. Read it as "at 100 s, this clock is good to 1e-11".
ACQ / DPLL / LOCK	The three stages of algorithm 10. ACQ pulls the frequency close using the counter; DPLL settles phase and frequency quickly; LOCK updates slowly and narrowly to approach minimum error.
BBR	Battery-backed RAM in a GNSS module — where a saved receiver configuration survives a power cut, if there is a backup cell.
DAC	Digital-to-analogue converter: the thing that turns a number into the control voltage. In this build it is usually not a chip at all — see PWM and dither.
DAC code / LSB	The number written to the control-voltage output, and one step of it. Every gain here is expressed per LSB, because that is the smallest move the loop can make.
dither	Deliberately varying the low bits of a PWM duty cycle from period to period so the *average* lands between two hardware steps. 13 hardware bits replayed over a 2048-entry table become about 24 effective bits.
EFC	Electronic frequency control — the OCXO's tuning input. Volts in, hertz out.
holdover	Running without GPS: the loop stops correcting and the OCXO free-runs on its last control voltage.
LTIC	Lars' time-interval counter — the ramp phase detector on PA1 (a capacitor charged between the PPS edge and the divided OCXO edge). "TIC" alone means the same detector.
NMEA	The plain-text sentences a GNSS module sends (`$GPRMC,…`): position, time, satellite count — everything except the binary configuration.
OCXO	Oven-controlled crystal oscillator: a crystal held at constant temperature, which is why it is stable and why it needs a warm-up.
picDIV	A small divider that turns 10 MHz into a 1 Hz edge for the phase detector. It must be *armed* (resynchronised) so its edge lands near the PPS.
PID / PI	The controller: **P** reacts to the error now, **I** to the error accumulated, **D** to how fast it is changing. Most loops here are PI — see Appendix B.
PPS	The one-pulse-per-second output of the GNSS receiver — the time reference everything is measured against.
PWM	Pulse-width modulation: a square wave whose duty cycle, after an RC filter, becomes the control voltage. Cheap, and with dither, better than most DAC chips.
qErr / sawtooth	The receiver's own estimate, in picoseconds, of how far its PPS edge missed true time. Timing receivers report it (`UBX-TIM-TP`); correcting for it removes a sawtooth-shaped error.
survey-in	A timing receiver measuring its own position for a long time so that it can then hold it fixed and spend all its solving on *time* instead.
Time Mode	The state a timing receiver enters after survey-in: position fixed, timing optimised. The display shows `HDOP:TIME`.
trend	The four-character word in the telemetry and on the display naming what the loop is doing right now: `ACQ`, `DPLL`, `LOCK`, `CORR`, `ZC`, `NOPH`, …
Vctl / Vphase	Vctl is the control voltage going *to* the oscillator; Vphase is the detector voltage coming *back* from the phase measurement. Two different pins, easy to confuse.
ZC	Zero-crossing cancellation (algorithm 12): removing a deliberate slew at the instant the phase crosses zero, leaving both frequency and phase correct at once.

Appendix D — The Kalman filter in plain words

Part 4.6 describes algorithm 13 by its knobs; this appendix describes it by its intuition. No formulas — a few pictures, that is all.

D.1 Three beliefs, and a pencil for each

The filter carries three beliefs: **how far the oscillator's phase is displaced** (in nanoseconds), **how fast that displacement is growing** (picoseconds per second), and **how that drift itself changes with age** (aging). With each belief it also carries the **thickness of a pencil**: the interval it knows it does not know. "Phase = 2 ns ± 1 ns" means: certain to one nanosecond — and over a night that certainty grows and shrinks on its own.

The pencils are half the filter. When a measurement arrives, the belief moves **toward it, but only as far as the measurement's pencil allows against the belief's own**. A confident measurement against a fuzzy belief — a big step. A fuzzy measurement against a confident belief — a light nudge. That is the whole "Kalman filter": belief with weights, renewed every second.

D.2 Two witnesses

- The **phase detector** is fresh but chatty: every second it reports the phase with ~2.5 ns of noise, and its zero wanders on its own (~2.6 ns over ~45 s). The filter measures that noise itself — from the differences between successive readings — and calls it R.
- The **TIM2 counter** is honest but blunt: it speaks frequency with a precision that only becomes useful after a hundred seconds. But it falls silent only when everything does — and it is what keeps the loop alive when the detector goes blind.

In short: over seconds the oscillator decides, over months GPS decides, and the filter picks the proportions **anew every second** — instead of once and forever, like algorithm 11's time constant.

D.3 The understanding and the hands: KT and KC

Since build 36 these are two separate knobs, and the difference is worth having:

- **KT** is the patience of the **understanding**: how widely the filter spreads belief across time, and how fast it may be allowed to conclude (it also sets the Q ceiling — the filter may not run faster than its horizon).
- **KC** is the speed of the **hands**: given that the filter already *knows* the phase sits 5 ns too far — over what time is that closed? Default KT/3 (KT/6 in builds 61–67), but only **after the first settle**: at a cold start the loop walks into band at the gentle KT pace (KT up to 100 s since build 67), and the quick hands switch themselves on once there is something quick to close. Faster hands mean a smaller *logged* phase error (ADEV ~23% better at mid tau) but more DAC dither — a budget, not a free lunch.
- **KF** is the hands **in a hurry**: when something has knocked the phase well out — a supply sag, a step of the crystal — they close it at 16 s until it has been home for two minutes, then go back to KC (build 69). Calm operation never sees it.

D.4 Healthy scepticism, or what the filter does when things go wrong

- **The 4σ gate.** A reading too wild for the filter's own predictions is refused — in singles and pairs at GPS events that is normal (the build 42 night: 320 rejects in 20 hours, longest run two). Ten in a row and the gate gives way: the state is wrong, not the data.
- **The trust test.** Phase and frequency are the same show seen from two sides: if the counter says frequency is being carried, the phase *must* be moving. A detector that does not move when it must is lying. The test ignores wobbles smaller than the counter's own resolution (so it cannot convict a healthy detector on the reference's own noise), and it asks what the frequency is doing *now*: the counter's average lags by up to a hundred seconds, so the loop adds back the corrections it has made since (build 65 — before that, a pull-in could convict a healthy detector on the motion of fifty seconds earlier). A convicted detector gets its voice back after half an hour, or at the next arm if its reading jumps with the divider, which a frozen one cannot do.

- **Silence after an arm.** Arming the divider stops it for a second, while the ramp keeps being sampled and reads the top rail (~1400 ns — it looks like a phase, but it is not one). For four seconds after an arm the filter simply does not hear the word "phase". Four seconds of `NOPH` right after `ARM` in a log are a sign of health, not a fault.
- **No phase without a reading.** After a restart or an arm, and after ten seconds without a reading, the filter keeps no phase of its own: it steers on frequency alone until the detector reads in band again. Until build 65 it carried a phase built from its own frequency estimate and steered to null it; on 07.10, started 0.48 Hz off with the detector against its rail, that drove the oscillator 0.8 Hz the other way. With no phase, a counter reading refused ten times in a row is believed rather than clipped — with nothing else to go on, a reading that keeps disagreeing is more likely right than the filter.
- **Below the window.** The detector's window is ±0.55 of `LC`'s span about its zero, and the display keeps to it. Algorithm 13 also reads below it, down to 0.9 of the span (never under 0.15 V): there the RC ramp is at its steepest and still single-valued, and that is where the divider lands after an arm. Such a reading is taken as fifteen times noisier than one at the zero (build 66 — before that, a landing below the window was no reading at all, and a re-arm lands where the last one did).
- **The edges count for less.** Past a quarter of the band from the zero, a reading is taken as noisier the further out it sits — ten times at the band's edge (build 66): out there the detector is noisier and its straight-line reading less true, and two readings 19 ns apart right after a landing once passed for a slope of 17 ns/s. A locked loop never sees this: it starts outside the band `LAT` sets.
- **A repeated reading.** When the firmware cannot read a burst at its edge it publishes the last reading again, and the trend says `KEPT`: the filter takes no phase from it, but the second is not a gap either. One second in a thousand at lock; a third of them during a pull-in is normal.
- **TIM2 silence after a restart.** For the first hundred seconds the counter's hundred-second average still carries the oscillator's warm-up; the filter waits for a clean measurement rather than believing history.

D.5 What a good night looks like

Trend `KAL` for ~99% of the time. Single `REJ`s at GPS events. `NOPH` only the four seconds after an `ARM`, `KEPT` about one second in a thousand. R holds ~1.3 ns on the author's board (build 64's night; older bench nights showed ~2.9), `sig` ~1 ns. `[at ceiling]` beside Q — on this bench the GPS micro-events press on the adaptation all night, so Q spends it at the ceiling; that is a property of the sky, not a fault. `KL` after the night adds the adaptation ratio and the Q water marks (`lo..hi`) — from which you can see whether Q ever left the ceiling.

D.6 When not to touch anything

The KR/KQ/KT/KC defaults are **measured from the board itself** — R from its detector, Q from its oscillator, the rest from the horizons. Pinning (`KR` to the white floor, `KQ` at the seed) is for comparative experiments, not daily work: every pin tells the filter it knows better than the board — and the board usually does not lie. If you want to see *whether* the filter is right, the best single habit remains: `KL`, `SW`, a one-hour log — and Appendix A, to know what to watch.